

Merge Sort

Split index ranges, finish both child calls, and merge their sorted results using protected temporary output.

How to use this chapter:

Learn the idea in plain language, read each diagram, predict every trace row, then study the commented Java and attempt the exam-style questions.

The whole process at a glance

Split index ranges, finish both child calls, and merge their sorted results using protected temporary output.

One card needs no sorting. Sort small piles first, then combine two sorted piles by taking their smallest remaining front card. Repeat until the original range is complete.

First split ranges until each contains one value. Then read the merge returns below in execution order: complete the left child, complete the right child, merge, return.

Green: sorted / found

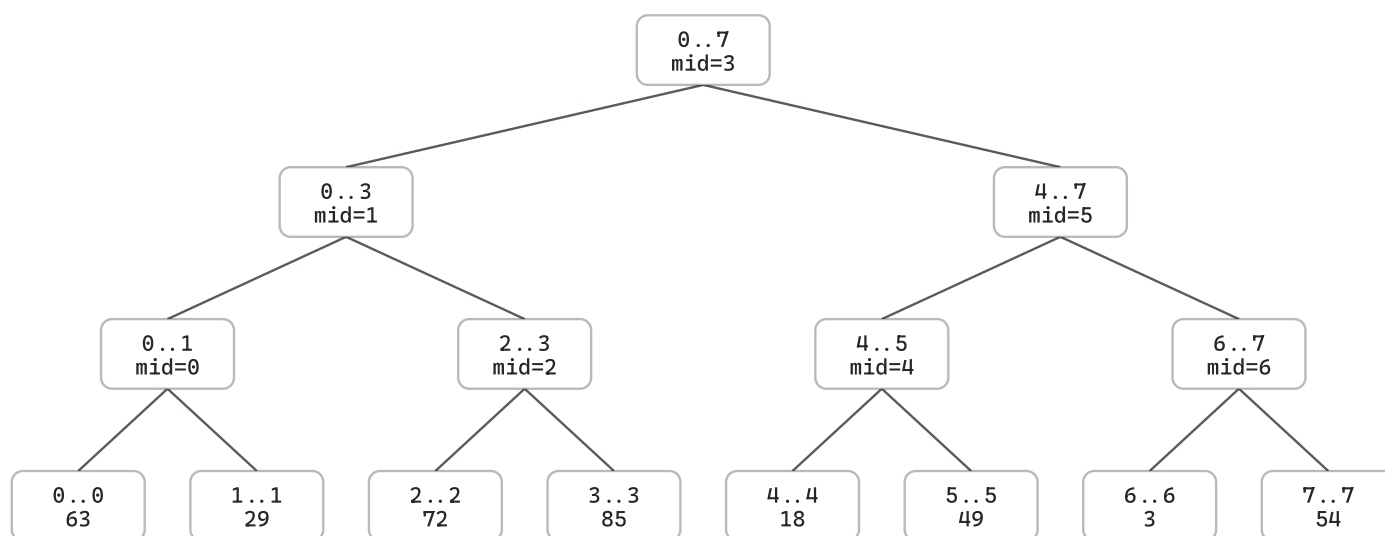
Blue: chosen value / midpoint

Orange: destination / held key

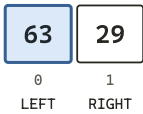
Crossed: ruled out

· = conceptual gap

Going down: follow the original split tree. Only range boundaries change. **Coming back:** read the seven numbered SVG merge returns below; each green result becomes an input to its parent.



Merge return 1: range 0..1



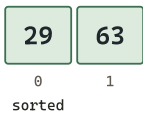
Children ready: LEFT [63] and RIGHT [29]. Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

1. $63 \leq 29 \rightarrow \text{false}$; take right and advance only its pointer.
2. Right run is exhausted. Copy a remaining left value without a key comparison.

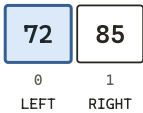
1 key comparisons; 2 temp writes and 2 copy-back writes.

↓ merge into temp → copy back → return



Sorted range 0..1 returns. Follow this result to the LEFT input of return 3 (range 0..3).

Merge return 2: range 2..3



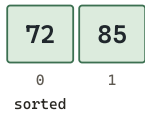
Children ready: LEFT [72] and RIGHT [85]. Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

1. $72 \leq 85 \rightarrow \text{true}$; take left and advance only its pointer.
2. Left run is exhausted. Copy a remaining right value without a key comparison.

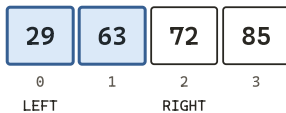
1 key comparisons; 2 temp writes and 2 copy-back writes.

↓ merge into temp → copy back → return



Sorted range 2..3 returns. Follow this result to the RIGHT input of return 3 (range 0..3).

Merge return 3: range 0..3



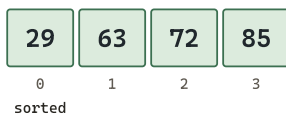
Children ready: LEFT [29, 63] and RIGHT [72, 85]. Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

1. $29 \leq 72 \rightarrow \text{true}$; take left and advance only its pointer.
2. $63 \leq 72 \rightarrow \text{true}$; take left and advance only its pointer.
3. Left run is exhausted. Copy a remaining right value without a key comparison.
4. Left run is exhausted. Copy a remaining right value without a key comparison.

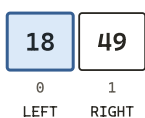
2 key comparisons; 4 temp writes and 4 copy-back writes.

↓ **merge into temp** → **copy back** → **return**



Sorted range 0..3 returns. Follow this result to the LEFT input of return 7 (range 0..7).

Merge return 4: range 4..5



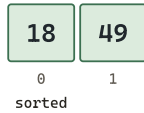
Children ready: LEFT [18] and RIGHT [49]. Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

1. $18 \leq 49 \rightarrow \text{true}$; take left and advance only its pointer.
2. Left run is exhausted. Copy a remaining right value without a key comparison.

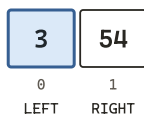
1 key comparisons; 2 temp writes and 2 copy-back writes.

↓ merge into temp → copy back → return



Sorted range 4..5 returns. Follow this result to the LEFT input of return 6 (range 4..7).

Merge return 5: range 6..7



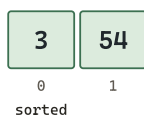
Children ready: LEFT [3] and RIGHT [54]. Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

1. $3 \leq 54 \rightarrow \text{true}$; take left and advance only its pointer.
2. Left run is exhausted. Copy a remaining right value without a key comparison.

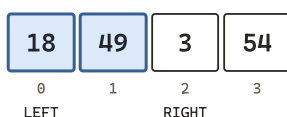
1 key comparisons; 2 temp writes and 2 copy-back writes.

↓ merge into temp → copy back → return



Sorted range 6..7 returns. Follow this result to the RIGHT input of return 6 (range 4..7).

Merge return 6: range 4..7



Children ready: LEFT [18, 49] and RIGHT [3, 54]. Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

1. $18 \leq 3 \rightarrow \text{false}$; take right and advance only its pointer.

2. $18 \leq 54 \rightarrow \text{true}$; take left and advance only its pointer.
3. $49 \leq 54 \rightarrow \text{true}$; take left and advance only its pointer.
4. Left run is exhausted. Copy a remaining right value without a key comparison.

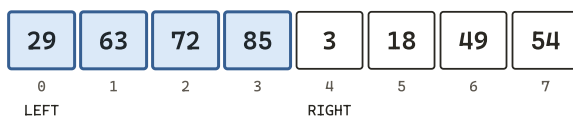
3 key comparisons; 4 temp writes and 4 copy-back writes.

↓ **merge into temp** → **copy back** → **return**



Sorted range 4..7 returns. Follow this result to the RIGHT input of return 7 (range 0..7).

Merge return 7: range 0..7



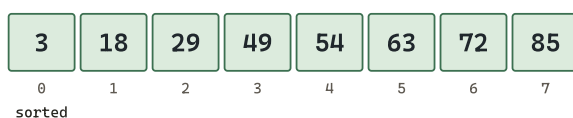
Children ready: LEFT [29, 63, 72, 85] and RIGHT [3, 18, 49, 54]. Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

1. $29 \leq 3 \rightarrow \text{false}$; take right and advance only its pointer.
2. $29 \leq 18 \rightarrow \text{false}$; take right and advance only its pointer.
3. $29 \leq 49 \rightarrow \text{true}$; take left and advance only its pointer.
4. $63 \leq 49 \rightarrow \text{false}$; take right and advance only its pointer.
5. $63 \leq 54 \rightarrow \text{false}$; take right and advance only its pointer.
6. Right run is exhausted. Copy a remaining left value without a key comparison.
7. Right run is exhausted. Copy a remaining left value without a key comparison.
8. Right run is exhausted. Copy a remaining left value without a key comparison.

5 key comparisons; 8 temp writes and 8 copy-back writes.

↓ **merge into temp** → **copy back** → **return**



Sorted range 0..7 returns. Root return: the entire array is sorted.

Sorted: [3, 18, 29, 49, 54, 63, 72, 85]. 14 key comparisons across all merges. 24 copy-back writes across three merge levels; peak extra array space is $O(n)$.

Sorting / Merge sort · The textbook definition

The textbook definition

Merge sort is a comparison-based, divide-and-conquer sorting algorithm. It splits the array into two halves, sorts each half by calling itself recursively, and then *merges* the two sorted halves into one sorted array. A sub-array of one value is already sorted, so it is the base case of the recursion. The work of actually putting values in order is done by the merge step on the way back up.

Divide and conquer Solve a big problem by solving smaller copies of the same problem, then combining the answers.	Run A sorted stretch of the array. Single values are runs of length 1; the merge step builds longer runs.
Split point mid The index where the range is cut: <code>mid = start + (end - start) / 2</code> . The left half is <code>start..mid</code> and the right half <code>mid + 1..end</code> .	Base case A range with one value (<code>start ≥ end</code>). It returns immediately, because one value is already sorted.
Merge Combine two sorted runs into one by repeatedly taking the smaller of the two front values.	Temporary arrays Copies of the two halves made during a merge, so values can be written back without overwriting unread ones.
Level One row of the recursion tree. Halving 8 values down to single values takes 3 splitting levels.	

Everything interesting about merge sort comes from one fact: **merging two sorted runs is cheap**, because only the two front values ever need to be compared. The recursion exists only to create sorted runs for the merge to work on.

In plain language

Merge sort turns “sort this pile” into “zip together two sorted piles” over and over.

Imagine two stacks of exam papers, each already sorted by mark from lowest to highest. You want one sorted stack. You do not need to look through the stacks: only the two papers on *top* matter. Whichever top paper has the lower mark goes onto the new stack, and then you look at the two tops again.

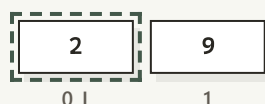
That is a **merge**. It is like a zip: two sorted lines go in, one sorted line comes out, and every paper is touched once.

Merge sort uses this trick on an unsorted pile: split the pile in half and keep splitting until each piece is a single paper. A single paper is a perfectly sorted stack of one. Now merge pairs of stacks, then pairs of those, until one stack is left.

Here is one merge in full: the sorted piles [2, 9] and [4, 5, 8]. Crossed-out cells have already been used; the labelled cell is the front of its pile.

Start: point at the front of each pile

LEFT RUN



RIGHT RUN

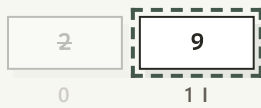


OUTPUT



After choice 1: took 2 from the left pile ($2 \leq 4$ was true)

LEFT RUN



RIGHT RUN

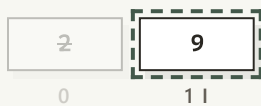


OUTPUT

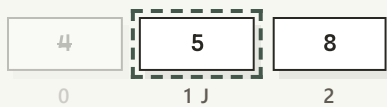


After choice 2: took 4 from the right pile ($9 \leq 4$ was false)

LEFT RUN



RIGHT RUN

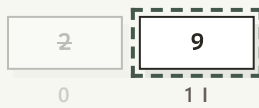


OUTPUT



After choice 3: took 5 from the right pile ($9 \leq 5$ was false)

LEFT RUN



RIGHT RUN

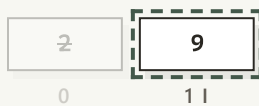


OUTPUT



After choice 4: took 8 from the right pile ($9 \leq 8$ was false)

LEFT RUN



RIGHT RUN

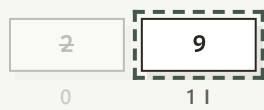


OUTPUT



The right pile is empty, so the leftover 9 is copied across

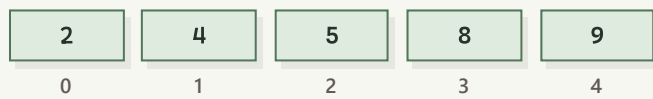
LEFT RUN



RIGHT RUN



OUTPUT



CHOICE	LEFT FRONT	RIGHT FRONT	COMPARE	TAKEN	OUTPUT SO FAR
1	2	4	$2 \leq 4$ is true	2 (from left)	[2]
2	9	4	$9 \leq 4$ is false	4 (from right)	[2, 4]
3	9	5	$9 \leq 5$ is false	5 (from right)	[2, 4, 5]
4	9	8	$9 \leq 8$ is false	8 (from right)	[2, 4, 5, 8]

Only **4 comparisons** were needed for 5 values (never more than $n - 1$), and the last value, 9, was copied without any comparison because the right pile had run out. Merging never looks at a value more than a couple of times: that is why it is cheap.

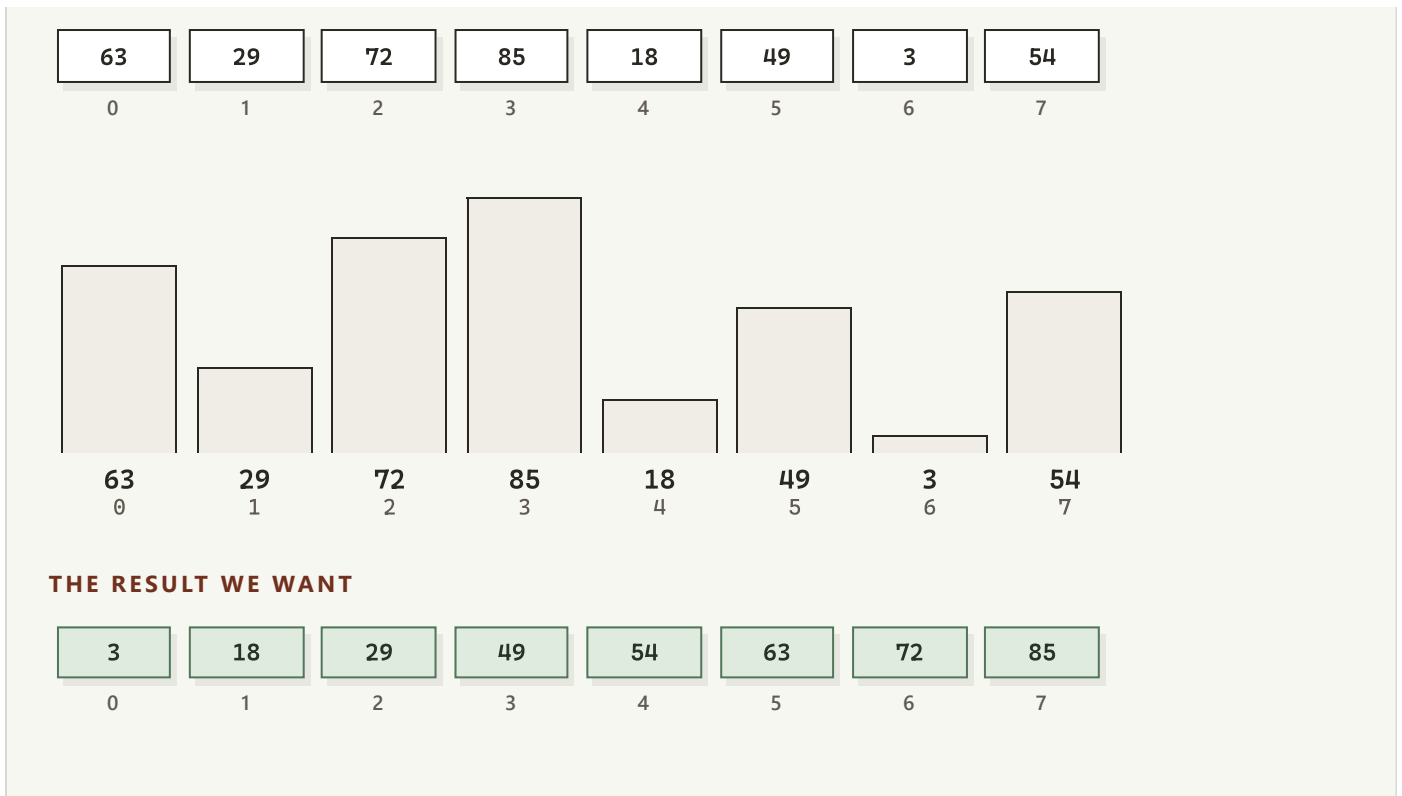
Detailed reference example: this part introduces its own input, shown below. The opening whole-run map and guided walkthrough keep their original input throughout; each later worked example states its new input explicitly.

Sorting / Merge sort · The array we will sort

The array we will sort

Eight values this time, so the array splits evenly all the way down: [63, 29, 72, 85, 18, 49, 3, 54].

STARTING ARRAY: EIGHT VALUES



For merge sort we use eight values, not five, and that is deliberate. Eight is $2 \times 2 \times 2$, so the array can be cut exactly in half three times ($8 \rightarrow 4 \rightarrow 2 \rightarrow 1$) and the picture of the whole algorithm is a perfectly balanced tree. Later examples use five and nine values, where the halves are uneven, so you will see that case too.

These are the values the lab uses, so the traces here match the Lab05 trace later in the guide.

Sorting / Merge sort · The main idea: divide, conquer, combine

The main idea: divide, conquer, combine

1

Divide: cut the range in half at `mid`.

2

Conquer: sort the left half, then the right half, by calling merge sort on each. These calls do the same thing again on smaller ranges.

3

Combine: merge the two sorted halves into one sorted range. This is the only step that moves values.

Splitting goes down, merging comes back up

Calling merge sort on the whole array makes two calls on halves, which make two calls each, until every call has a single value. Nothing is sorted on the way *down*; the array is only being cut up. Here is the cut-up for the eight values, with each box drawn as wide as the number of values it holds.

level 0	63 29 72 85 18 49 3 54							
level 1	63 29 72 85				18 49 3 54			
level 2	63 29		72 85		18 49		3 54	
level 3	63	29	72	85	18	49	3	54

Now the calls return. Each call, once both of its halves are sorted, merges them. Read the next picture from the **bottom row upward**: the bottom row (single values) is already sorted; each row above is made by merging pairs of boxes from the row below, until the top box is the whole sorted array.

level 0	3 18 29 49 54 63 72 85							
level 1	29 63 72 85				3 18 49 54			
level 2	29 63		72 85		18 49		3 54	
level 3	63	29	72	85	18	49	3	54

Why this is fast

Look at any one row of the tree: all its boxes together hold the 8 values once, so merging a whole row costs about 8 steps. There are only 3 rows of merges, because halving 8 down to 1 takes 3 cuts. Total work $\approx 8 \times 3 = 24$ value-writes. For a million values there are only about 20 rows, so the work is about one million $\times$ 20, not one million $\times$ one million. That halving is the same trick binary search uses.

The mental picture: the recursion is just a way of *organising* merges. The real sorting happens in the merge step, always on two runs that are already sorted.

Sorting / Merge sort · Split, call, merge

Split, call, merge

Three different things happen in merge sort, and a trace mixes them up unless you keep them apart.

Split Calculating <code>mid</code> . No values move; only a number is computed.	Call Recurring on a half. A call that waits for its two halves to finish is parked on the <i>call stack</i> .
Merge The only phase that moves values: they are copied from two temporary arrays back into <code>arr</code> in sorted order.	Comparison One question per output value while both runs still have values: <code>left[i] ≤ right[j]</code> .

Where exactly is the split?

The left half is `start..mid` and the right half `mid + 1..end`, with `mid = start + (end - start) / 2` (integer division). When the number of values is odd, the left half gets the extra value:

RANGE	VALUES	MID	LEFT HALF	RIGHT HALF
(0, 7)	8	$0 + (7 - 0) / 2 = 3$	0..3 (4 values)	4..7 (4 values)
(0, 3)	4	$0 + (3 - 0) / 2 = 1$	0..1 (2 values)	2..3 (2 values)
(4, 7)	4	$4 + (7 - 4) / 2 = 5$	4..5 (2 values)	6..7 (2 values)
(0, 4)	5	$0 + (4 - 0) / 2 = 2$	0..2 (3 values)	3..4 (2 values)
(5, 8)	4	$5 + (8 - 5) / 2 = 6$	5..6 (2 values)	7..8 (2 values)

What is waiting on the call stack?

Merge sort calls itself, so Java keeps a stack of calls that have started but not finished. At the moment of the *first* merge on our eight values, three calls are alive:

POSITION	CALL	WHAT IT IS DOING
top (running)	<code>mergeSort(arr, 0, 1)</code>	Both of its halves, (0, 0) and (1, 1), have returned: it merges them now.
below	<code>mergeSort(arr, 0, 3)</code>	Waiting. Its left half (0, 1) is still running; the right half (2, 3) has not started.
bottom	<code>mergeSort(arr, 0, 7)</code>	Waiting. Its left half (0, 3) is still running; the right half (4, 7) has not started.

The stack is never deeper than the number of levels (here 4), which is why merge sort's recursion cannot overflow the stack for any realistic array: $\log_2 n$ is small even for a billion values (about 30).

The left half is always finished first

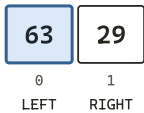
Because the left call is written before the right call, the whole left side is sorted and merged before the right side is even cut up. This is why the merges happen in *depth-first* order (see the trace table earlier), and why a bottom-up version that goes level by level makes the same merges in a different order.

Guided walkthrough: predict the next major step

Predict the next completed merge, then advance. Each scene keeps all its movements together.

Guided walkthrough: major steps

Merge return 1: range 0..1



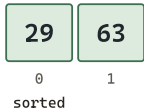
Children ready: LEFT [63] and RIGHT [29]. Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

- 1. $63 \leq 29 \rightarrow \text{false}$; take right and advance only its pointer.
- 2. Right run is exhausted. Copy a remaining left value without a key comparison.

1 key comparisons; 2 temp writes and 2 copy-back writes.

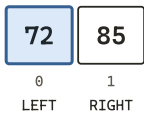
↓ merge into temp → copy back → return



Sorted range 0..1 returns. Follow this result to the LEFT input of return 3 (range 0..3).

Both child ranges are sorted before this merge. Return [29,63].

Merge return 2: range 2..3



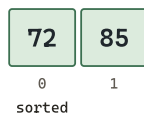
Children ready: LEFT [72] and RIGHT [85]. Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

1. $72 \leq 85 \rightarrow \text{true}$; take left and advance only its pointer.
2. Left run is exhausted. Copy a remaining right value without a key comparison.

1 key comparisons; 2 temp writes and 2 copy-back writes.

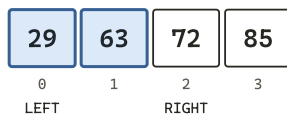
↓ merge into temp → copy back → return



Sorted range 2..3 returns. Follow this result to the RIGHT input of return 3 (range 0..3).

Both child ranges are sorted before this merge. Return [72,85].

Merge return 3: range 0..3



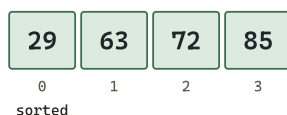
Children ready: LEFT [29, 63] and RIGHT [72, 85]. Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

1. $29 \leq 72 \rightarrow \text{true}$; take left and advance only its pointer.
2. $63 \leq 72 \rightarrow \text{true}$; take left and advance only its pointer.
3. Left run is exhausted. Copy a remaining right value without a key comparison.
4. Left run is exhausted. Copy a remaining right value without a key comparison.

2 key comparisons; 4 temp writes and 4 copy-back writes.

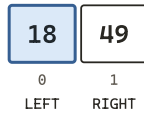
↓ merge into temp → copy back → return



Sorted range 0..3 returns. Follow this result to the LEFT input of return 7 (range 0..7).

Both child ranges are sorted before this merge. Return [29,63,72,85].

Merge return 4: range 4..5



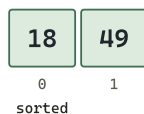
Children ready: LEFT [18] and RIGHT [49].
Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

1. $18 \leq 49 \rightarrow \text{true}$; take left and advance only its pointer.
2. Left run is exhausted. Copy a remaining right value without a key comparison.

1 key comparisons; 2 temp writes and 2 copy-back writes.

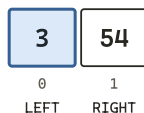
↓ merge into temp → copy back → return



Sorted range 4..5 returns. Follow this result to the LEFT input of return 6 (range 4..7).

Both child ranges are sorted before this merge. Return [18,49].

Merge return 5: range 6..7



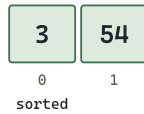
Children ready: LEFT [3] and RIGHT [54].
Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

1. $3 \leq 54 \rightarrow \text{true}$; take left and advance only its pointer.
2. Left run is exhausted. Copy a remaining right value without a key comparison.

1 key comparisons; 2 temp writes and 2 copy-back writes.

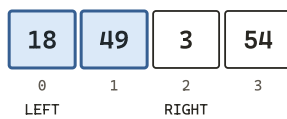
↓ merge into temp → copy back → return



Sorted range 6..7 returns. Follow this result to the RIGHT input of return 6 (range 4..7).

Both child ranges are sorted before this merge. Return [3,54].

Merge return 6: range 4..7



Children ready: LEFT [18, 49] and RIGHT [3, 54]. Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

1. $18 \leq 3 \rightarrow \text{false}$; take right and advance only its pointer.
2. $18 \leq 54 \rightarrow \text{true}$; take left and advance only its pointer.
3. $49 \leq 54 \rightarrow \text{true}$; take left and advance only its pointer.
4. Left run is exhausted. Copy a remaining right value without a key comparison.

3 key comparisons; 4 temp writes and 4 copy-back writes.

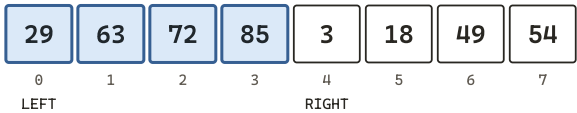
↓ merge into temp → copy back → return



Sorted range 4..7 returns. Follow this result to the RIGHT input of return 7 (range 0..7).

Both child ranges are sorted before this merge. Return [3,18,49,54].

Merge return 7: range 0..7



Children ready: LEFT [29, 63, 72, 85] and RIGHT [3, 18, 49, 54]. Compare only the front unread values; copy the smaller to temp and advance its pointer.

▼ Follow each comparison and copy

- 1. 29 <= 3 → false; take right and advance only its pointer.
- 2. 29 <= 18 → false; take right and advance only its pointer.
- 3. 29 <= 49 → true; take left and advance only its pointer.
- 4. 63 <= 49 → false; take right and advance only its pointer.
- 5. 63 <= 54 → false; take right and advance only its pointer.
- 6. Right run is exhausted. Copy a remaining left value without a key comparison.
- 7. Right run is exhausted. Copy a remaining left value without a key comparison.
- 8. Right run is exhausted. Copy a remaining left value without a key comparison.

5 key comparisons; 8 temp writes and 8 copy-back writes.

↓ merge into temp → copy back → return



Sorted range 0..7 returns. Root return: the entire array is sorted.

Both child ranges are sorted before this merge. Return [3,18,29,49,54,63,72,85].

▼ Full trace table - select a row to jump to its scene

1	0..1	[63] + [29]	[29,63]	1
2	2..3	[72] + [85]	[72,85]	1

3	0..3	[29,63] + [72,85]	[29,63,72,85]	2
4	4..5	[18] + [49]	[18,49]	1
5	6..7	[3] + [54]	[3,54]	1
RETURN	RANGE	SORTED INPUTS	RESULT	COMPARISONS
6	4..7	[18,49] + [3,54]	[3,18,49,54]	3
7	0..7	[29,63,72,85] + [3,18,49,54]	[3,18,29,49,54,63,72,85]	5

RETURN	RANGE	SORTED INPUTS	RESULT	COMPARISONS
--------	-------	---------------	--------	-------------

The trace at a glance

RETURN	RANGE	SORTED INPUTS	RESULT	COMPARISONS
1	0..1	[63] + [29]	[29,63]	1
2	2..3	[72] + [85]	[72,85]	1
3	0..3	[29,63] + [72,85]	[29,63,72,85]	2
4	4..5	[18] + [49]	[18,49]	1
5	6..7	[3] + [54]	[3,54]	1
6	4..7	[18,49] + [3,54]	[3,18,49,54]	3
7	0..7	[29,63,72,85] + [3,18,49,54]	[3,18,29,49,54,63,72,85]	5

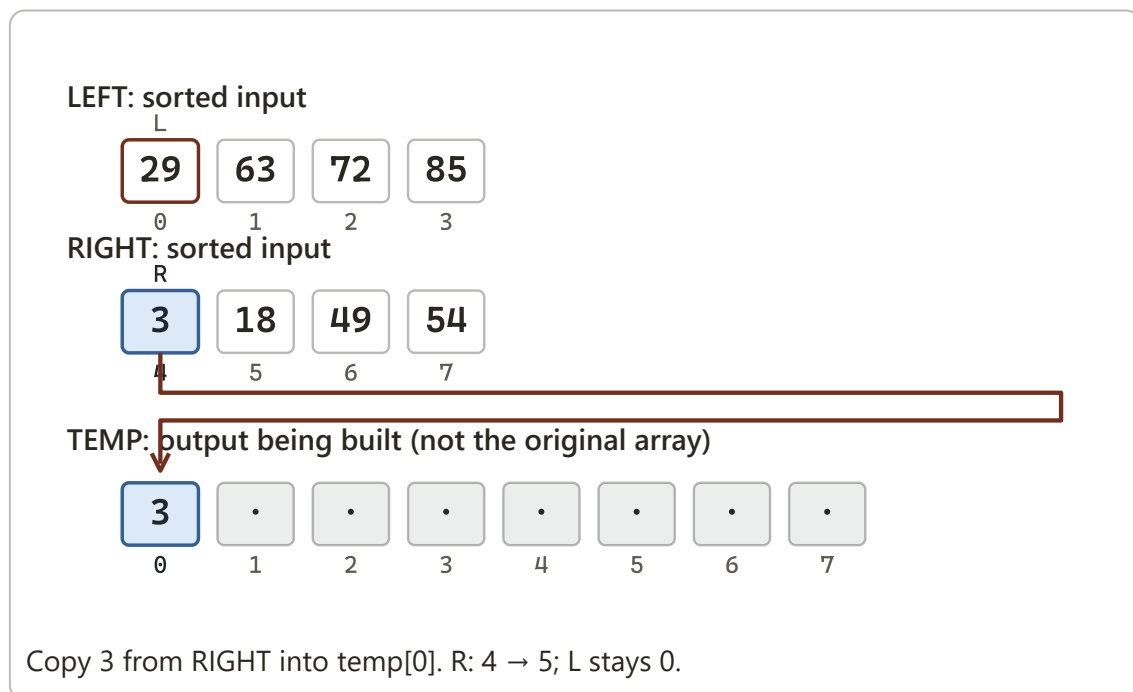
A closer look at one merge return

Merging is **not swapping** the two inputs. Read only the front unread item from each sorted run, copy the smaller into a separate output, and advance only that input pointer. The other pointer waits.

For this final merge, LEFT is [29, 63, 72, 85] and RIGHT is [3, 18, 49, 54]. The arrow connects the chosen input to its new output position. Faded input cells have already been consumed.

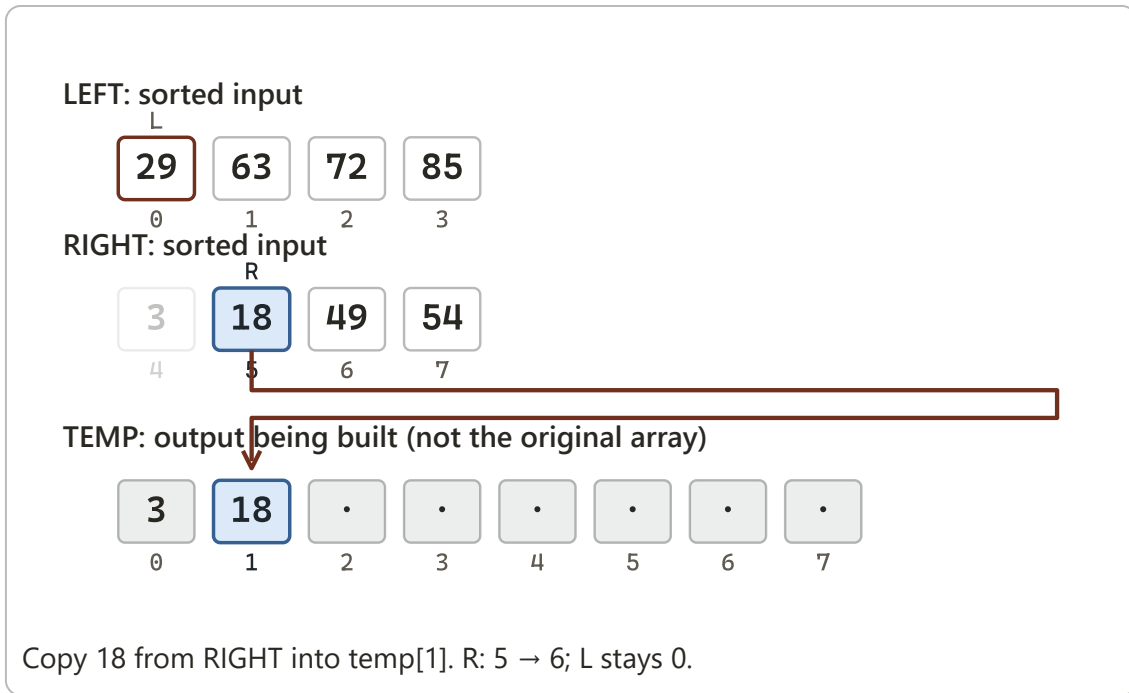
Follow one value from input to output

Output 1 of 8: Write temp[0] = 3



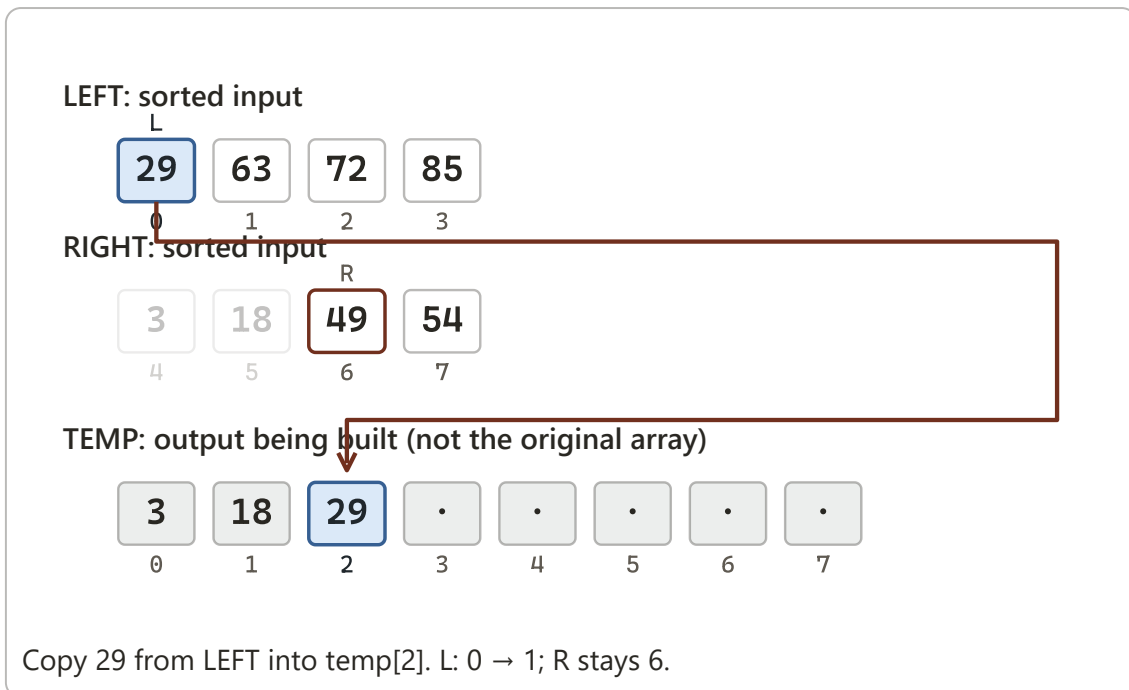
29 ≤ 3 → false; take right and advance only its pointer.

Output 2 of 8: Write temp[1] = 18



29 <= 18 → false; take right and advance only its pointer.

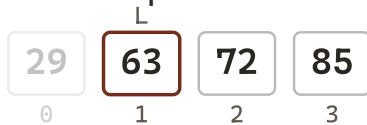
Output 3 of 8: Write temp[2] = 29



29 <= 49 → true; take left and advance only its pointer.

Output 4 of 8: Write temp[3] = 49

LEFT: sorted input



RIGHT: sorted input



TEMP: output being built (not the original array)

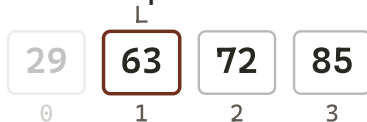


Copy 49 from RIGHT into temp[3]. R: 6 → 7; L stays 1.

63 <= 49 → false; take right and advance only its pointer.

Output 5 of 8: Write temp[4] = 54

LEFT: sorted input



RIGHT: sorted input



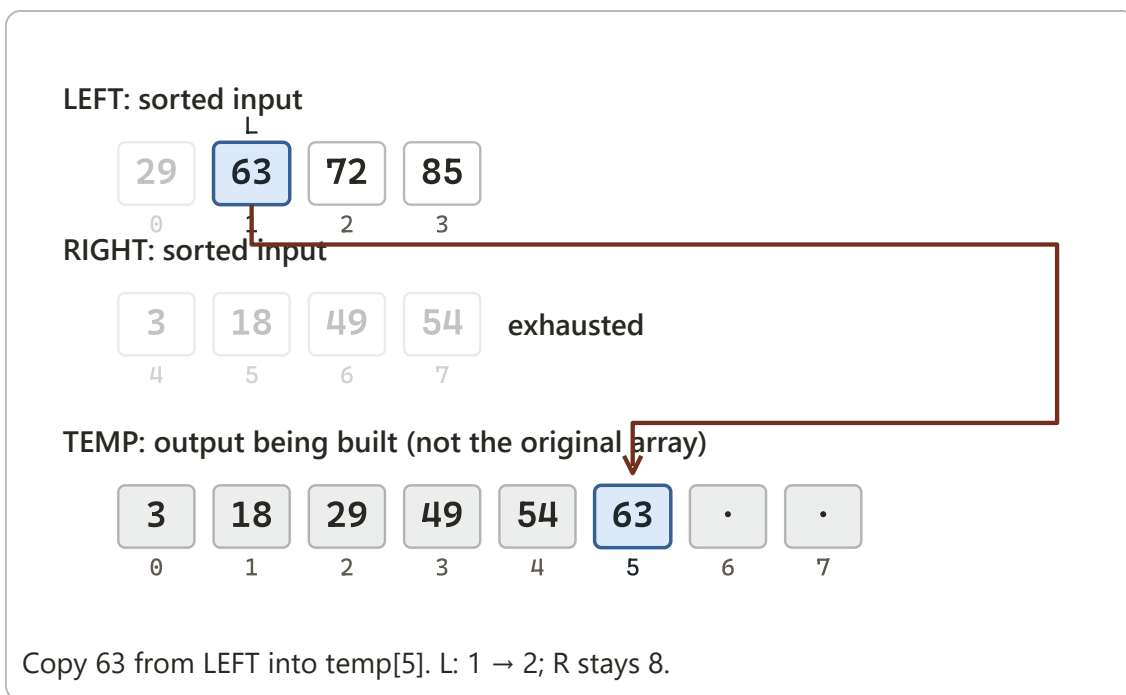
TEMP: output being built (not the original array)



Copy 54 from RIGHT into temp[4]. R: 7 → 8; L stays 1.

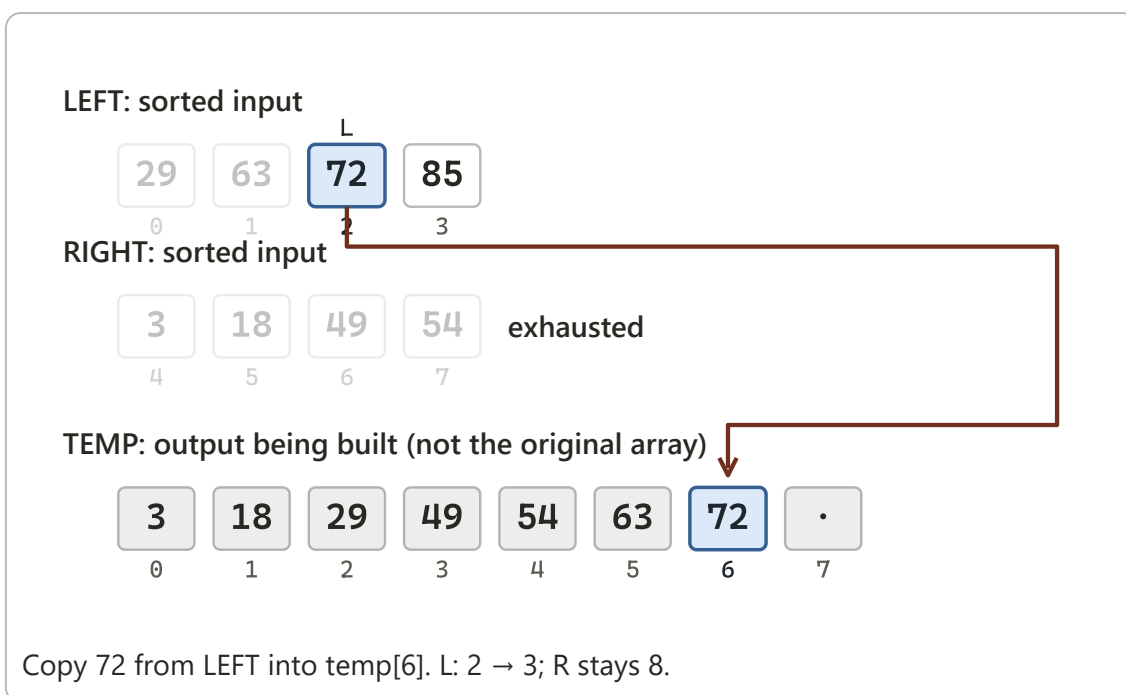
63 <= 54 → false; take right and advance only its pointer.

Output 6 of 8: Copy left leftover 63



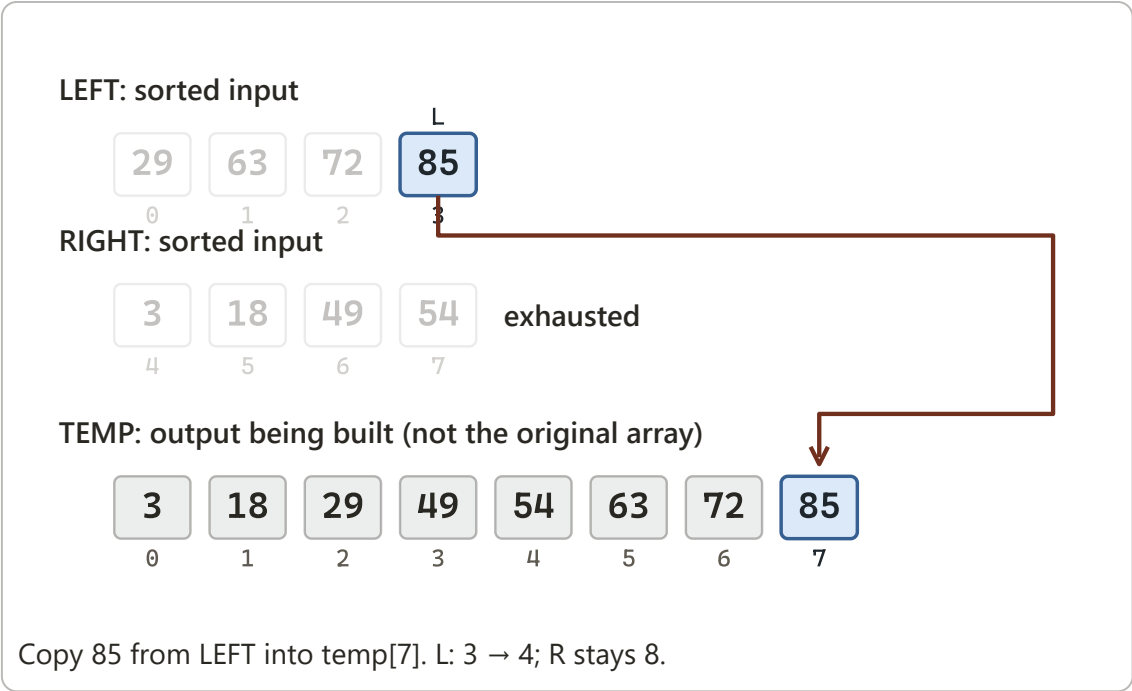
Right run is exhausted. Copy a remaining left value without a key comparison.

Output 7 of 8: Copy left leftover 72



Right run is exhausted. Copy a remaining left value without a key comparison.

Output 8 of 8: Copy left leftover 85



Right run is exhausted. Copy a remaining left value without a key comparison.

▼ Full trace table - select a row to jump to its scene

1	3	0	1	[3]
2	18	0	2	[3, 18]
3	29	1	2	[3, 18, 29]
4	49	1	3	[3, 18, 29, 49]
5	54	1	4	[3, 18, 29, 49, 54]
6	63	2	4	[3, 18, 29, 49, 54, 63]
7	72	3	4	[3, 18, 29, 49, 54, 63, 72]

STEP	WRITTEN VALUE	LEFT CONSUMED	RIGHT CONSUMED	OUTPUT
8	85	4	4	[3, 18, 29, 49, 54, 63, 72, 85]

After the last choice: temp is [3, 18, 29, 49, 54, 63, 72, 85]. Copy temp[k] to arr[start + k] for every k. Only then is the original range sorted and this call finished. Copying leftovers does not compare two keys; copying back is not a swap.

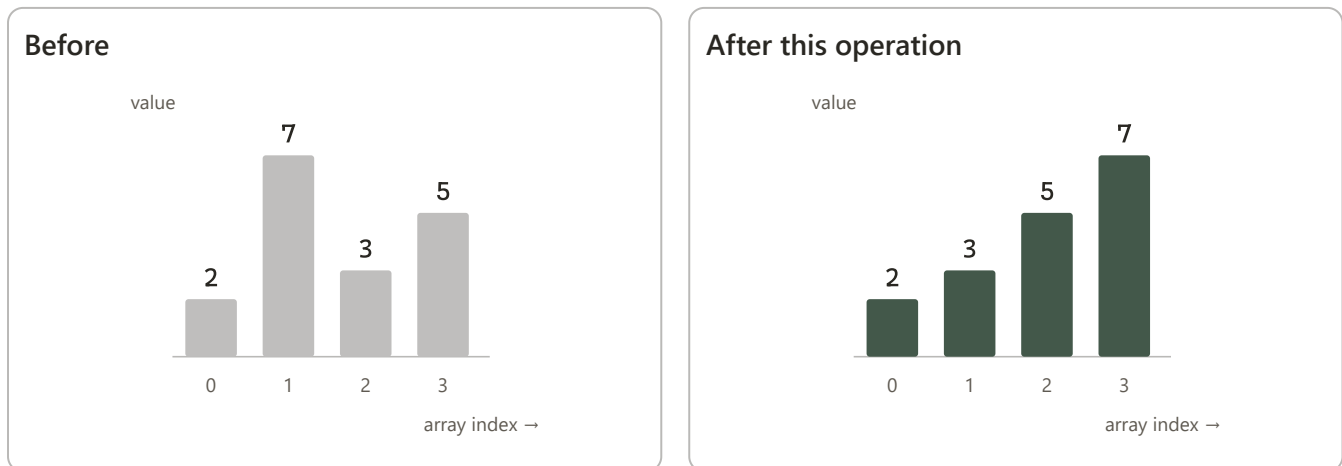
Why only the fronts? Everything behind a run's front is at least that large. Therefore the smaller of the two fronts is the smallest unread value overall. On equality, taking LEFT first preserves stability.

Read the algorithm as a bar graph

Bar height means **value**; horizontal position means **index**. All panels below use the same height scale. A taller bar is not

STEP	WRITTEN VALUE	LEFT CONSUMED	RIGHT CONSUMED	OUTPUT
------	---------------	---------------	----------------	--------

Two rising runs become one rising run



LEFT [2,7] and RIGHT [3,5] are individually sorted. Compare $2 \leq 3$: take 2. Compare $7 \leq 3$: take 3. Compare $7 \leq 5$: take 5. RIGHT is empty, so copy leftover 7. Output [2,3,5,7] was built separately, then copied back. Three key comparisons, four temporary writes, four copy-back writes.

Check yourself: identify which indexes are guaranteed final, which values were merely inspected, and whether the operation swapped, shifted or copied data. A sorted ascending array has non-decreasing heights, including equal-height neighbours.

Sorting / Merge sort

Divide first. Merge on the way back.

Merge sort repeatedly divides an array into halves until every piece has one value. It then combines sorted pieces into larger sorted pieces.

1

Base case: $\text{start} \geq \text{end}$. A section with one value is already sorted.

2

Divide: calculate `mid`.

3

Recur left: sort `start .. mid`.

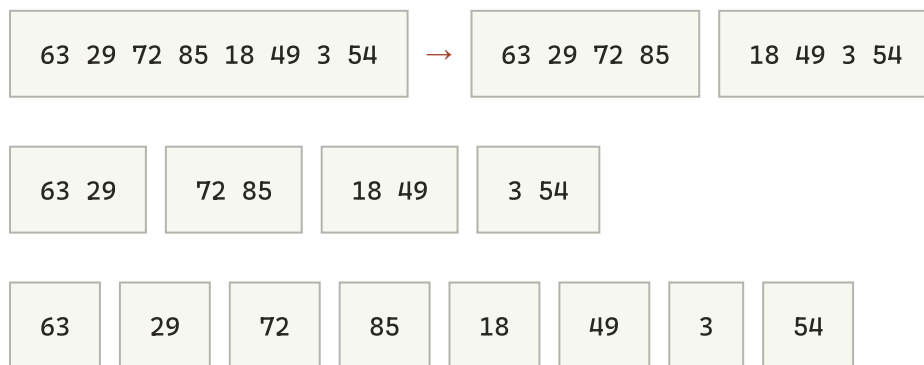
4

Recur right: sort `mid+1 .. end`.

5

Merge: combine both sorted halves.

Illustration · split the example array



Merge phase · the same tree level by level (the recursion itself merges depth-first: see the trace table below)



29 63 72 85

3 18 49 54

← two merges of pairs

3 18 29 49 54 63 72 85

← one final merge

Call tree · mergeSort(arr, 0, 7)

The calls run top to bottom in this order. Each call goes left all the way down before it goes right, and its merge happens only after both of its recursive calls have returned. Indentation shows the depth.

CALL	CALL MADE	MID	WHAT HAPPENS
1	mergeSort(arr, 0, 7)	3	Call left (0, 3), then right (4, 7), then merge
2	mergeSort(arr, 0, 3)	1	Call left (0, 1), then right (2, 3), then merge
3	mergeSort(arr, 0, 1)	0	Call left (0, 0), then right (1, 1), then merge
4	mergeSort(arr, 0, 0)	—	0 ≥ 0: one value, return
5	mergeSort(arr, 1, 1)	—	1 ≥ 1: one value, return
6	mergeSort(arr, 2, 3)	2	Call left (2, 2), then right (3, 3), then merge
7	mergeSort(arr, 2, 2)	—	2 ≥ 2: one value, return
8	mergeSort(arr, 3, 3)	—	3 ≥ 3: one value, return
9	mergeSort(arr, 4, 7)	5	Call left (4, 5), then right (6, 7), then merge
10	mergeSort(arr, 4, 5)	4	Call left (4, 4), then right (5, 5), then merge
11	mergeSort(arr, 4, 4)	—	4 ≥ 4: one value, return
12	mergeSort(arr, 5, 5)	—	5 ≥ 5: one value, return
13	mergeSort(arr, 6, 7)	6	Call left (6, 6), then right (7, 7), then merge
14	mergeSort(arr, 6, 6)	—	6 ≥ 6: one value, return
15	mergeSort(arr, 7, 7)	—	7 ≥ 7: one value, return

Trace table · merge on return

Each merge fires as the calls above return, so the order is depth-first, *not* level by level: the left half is fully sorted before the right half is even split.

RETURN STEP	MERGE CALL	RETURNS FROM CALL #	LEFT SORTED HALF	RIGHT SORTED HALF	MERGED RESULT
1	merge(arr, 0, 0, 1)	call 3	[63]	[29]	[29, 63]
2	merge(arr, 2, 2, 3)	call 6	[72]	[85]	[72, 85]
3	merge(arr, 0, 1, 3)	call 2	[29, 63]	[72, 85]	[29, 63, 72, 85]
4	merge(arr, 4, 4, 5)	call 10	[18]	[49]	[18, 49]
5	merge(arr, 6, 6, 7)	call 13	[3]	[54]	[3, 54]
6	merge(arr, 4, 5, 7)	call 9	[18, 49]	[3, 54]	[3, 18, 49, 54]
7	merge(arr, 0, 3, 7)	call 1	[29, 63, 72, 85]	[3, 18, 49, 54]	[3, 18, 29, 49, 54, 63, 72, 85]

Implementation

Recursive merge sort

```
public static void mergeSort(int[] arr, int start, int end) {
    // A zero- or one-value range is already sorted.
    if (start < end) {
        // Split without risking integer overflow.
        int mid = start + (end - start) / 2;
        // Sort both halves before combining them.
        mergeSort(arr, start, mid);
        mergeSort(arr, mid + 1, end);
        merge(arr, start, mid, end);
    }
}
```

▼ The merge method used by Lab05

Java

```
public static void merge(int[] arr, int s, int m, int e) {
    // Requires: import java.util.Arrays;
    // Copy both already-sorted halves into temporary arrays.
    int[] left = Arrays.copyOfRange(arr, s, m + 1);
    int[] right = Arrays.copyOfRange(arr, m + 1, e + 1);
    int i = 0, j = 0, cur = s;

    // Copy the smaller front value back into the original array.
    while (i < left.length && j < right.length)
        arr[cur++] = left[i] ≤ right[j] ? left[i++] : right[j++];
    // Exactly one of these leftover loops will copy values.
    while (i < left.length) arr[cur++] = left[i++];
    while (j < right.length) arr[cur++] = right[j++];
}
```

▼ Iterative bottom-up merge sort

Java

```
public static void mergeSortIterative(int[] arr) {
    // width is the current size of each sorted run.
    for (int width = 1; width < arr.length; width *= 2) {
        // Merge neighbouring runs of size width.
        for (int start = 0; start < arr.length - 1; start += 2 * width) {
            int mid = Math.min(start + width - 1, arr.length - 1);
            int end = Math.min(start + 2 * width - 1, arr.length - 1);
            if (mid < end) merge(arr, start, mid, end);
        }
    }
}
```

Bottom-up trace on [63, 29, 72, 85, 18, 49, 3, 54]. The outer loop doubles width; the inner loop merges neighbouring runs of that width.

WIDTH	RUNS MERGED	ARRAY AFTER THIS PASS
1	[63]+[29], [72]+[85], [18]+[49], [3]+[54]	[29, 63, 72, 85, 18, 49, 3, 54]

WIDTH	RUNS MERGED	ARRAY AFTER THIS PASS
2	[29,63]+[72,85], [18,49]+[3,54]	[29, 63, 72, 85, 3, 18, 49, 54]
4	[29,63,72,85]+[3,18,49,54]	[3, 18, 29, 49, 54, 63, 72, 85]

These are the same seven merges as the recursive trace table, performed in a different order: bottom-up goes level by level, while the recursive version finishes the whole left half before touching the right. When n is not a power of two, `Math.min` clamps the last run, and the `mid < end` test skips a run that has no partner until a wider pass reaches it.

Write it from scratch

- 1 Stop on a one-value range.
- 2 Calculate the midpoint.
- 3 Sort the left half.
- 4 Sort the right half.
- 5 Merge both sorted halves.

Ascending ↔ descending

Ascending merge chooses the left value when `left[i] ≤ right[j]`. Descending chooses it when `left[i] ≥ right[j]`.

Same fronts, opposite choice: with left front 29 and right front 3, ascending takes 3; descending takes 29 because `29 ≥ 3`.

Debug checklist

- Include `m + 1` in the left copy's exclusive end.
- Recur on `mid + 1` for the right half.
- Copy leftovers after the paired comparison loop.

Worked example · final merge

Merge [29, 63, 72, 85] and [3, 18, 49, 54] into one sorted run. `i` and `j` point at the front of each run; the lower-front value moves to the output, and its pointer advances. Cells crossed out have already been copied.

Start · both runs are sorted; i and j mark the front of each run

LEFT RUN



RIGHT RUN



OUTPUT

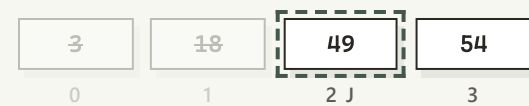


After 2 choices · i = 0, j = 2 (3 and 18 came from the right run)

LEFT RUN



RIGHT RUN

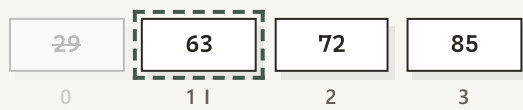


OUTPUT



After 5 choices · the right run is exhausted (j = 4)

LEFT RUN



RIGHT RUN



OUTPUT



Leftovers copied · 63, 72, 85 move across unchanged

LEFT RUN



RIGHT RUN



OUTPUT



OUTPUT INDEX	I	J	LEFT[I]	RIGHT[J]	CHOSEN	REASON
0	0	0	29	3	3	3 < 29
1	0	1	29	18	18	18 < 29
2	0	2	29	49	29	29 ≤ 49
3	1	2	63	49	49	49 < 63
4	1	3	63	54	54	54 < 63

OUTPUT INDEX	I	J	LEFT[I]	RIGHT[J]	CHOSEN	REASON
5..7	1..3	4	63, 72, 85	empty	63, 72, 85	Right run is empty; copy the leftovers

Result: [3, 18, 29, 49, 54, 63, 72, 85] after 5 comparisons and 3 leftover copies. Eight values were written to the output.

Why $O(n \log n)$? There are about $\log_2 n$ levels of halving, and merging touches all n values at every level.

Counting it for the 8-value array. Every level re-merges all 8 values, and halving 8 values down to single values takes 3 levels.

LEVEL (BOTTOM TO TOP)	MERGES AT THIS LEVEL	RUN SIZES MERGED	VALUES WRITTEN
1	4	1 + 1	8
2	2	2 + 2	8
3	1	4 + 4	8
Total	—	—	$24 = 8 \times \log_2 8$

This particular run made only 14 comparisons because leftovers are copied without comparing. The growth rate is still $n \log_2 n$.

BEST

$O(n \log n)$

AVERAGE

$O(n \log n)$

WORST

$O(n \log n)$

STABLE

Yes

IN-PLACE

No*

ADAPTIVE

No

AUXILIARY SPACE

$O(n)$

CALL DEPTH

$O(\log n)$

* Merge sort needs temporary copies of the halves, so it uses $O(n)$ auxiliary space and is not in-place. In-place merge variants exist but are far more complicated and are outside this guide.

Questions

▼ What is the base case?

`start` $\geq$ `end`. A range containing zero or one item is already sorted.

▼ Why does merge sort need extra memory?

The merge step temporarily copies the left and right halves so their smallest remaining values can be compared safely.

Merge sort in one sentence

Divide the range into one-value pieces, then merge sorted halves while choosing the smaller front value.

Fully annotated Java: connect each line to the movement

java

```
public static void mergeSort(int[] arr, int start, int end) {
    // Inclusive range of zero or one value: stop before splitting.
    if (start ≥ end) return;
    // Split INDEX RANGES. This statement does not move array values.
    int mid = start + (end - start) / 2;
    mergeSort(arr, start, mid);    // Finish the entire left subtree first.
    mergeSort(arr, mid + 1, end);  // Then finish the entire right subtree.
    // Both halves are now sorted: the merge precondition is satisfied.
    merge(arr, start, mid, end);
}

private static void merge(int[] arr, int start, int mid, int end) {
    // Protect unread input: never merge directly over unprotected values.
    int[] temp = new int[end - start + 1];
    int left = start;    // First unread value of left run start..mid.
    int right = mid + 1; // First unread value of right run mid+1..end.
    int out = 0;         // Next free position in temp (starts at zero).
    while (left ≤ mid && right ≤ end) {
        // Left-first ties preserve the order of equal records.
        if (arr[left] ≤ arr[right]) {
            temp[out] = arr[left]; left++;
        } else {
            temp[out] = arr[right]; right++;
        }
        out++; // Exactly one output value was written.
    }
    // One run is exhausted. Its partner's suffix is already sorted.
    while (left ≤ mid) { temp[out++] = arr[left++]; }
    while (right ≤ end) { temp[out++] = arr[right++]; }
    // Offset matters: temp[0] belongs at arr[start], not always arr[0].
    for (int k = 0; k < temp.length; k++) arr[start + k] = temp[k];
    // First call: mergeSort(arr, 0, arr.length - 1).
}
```

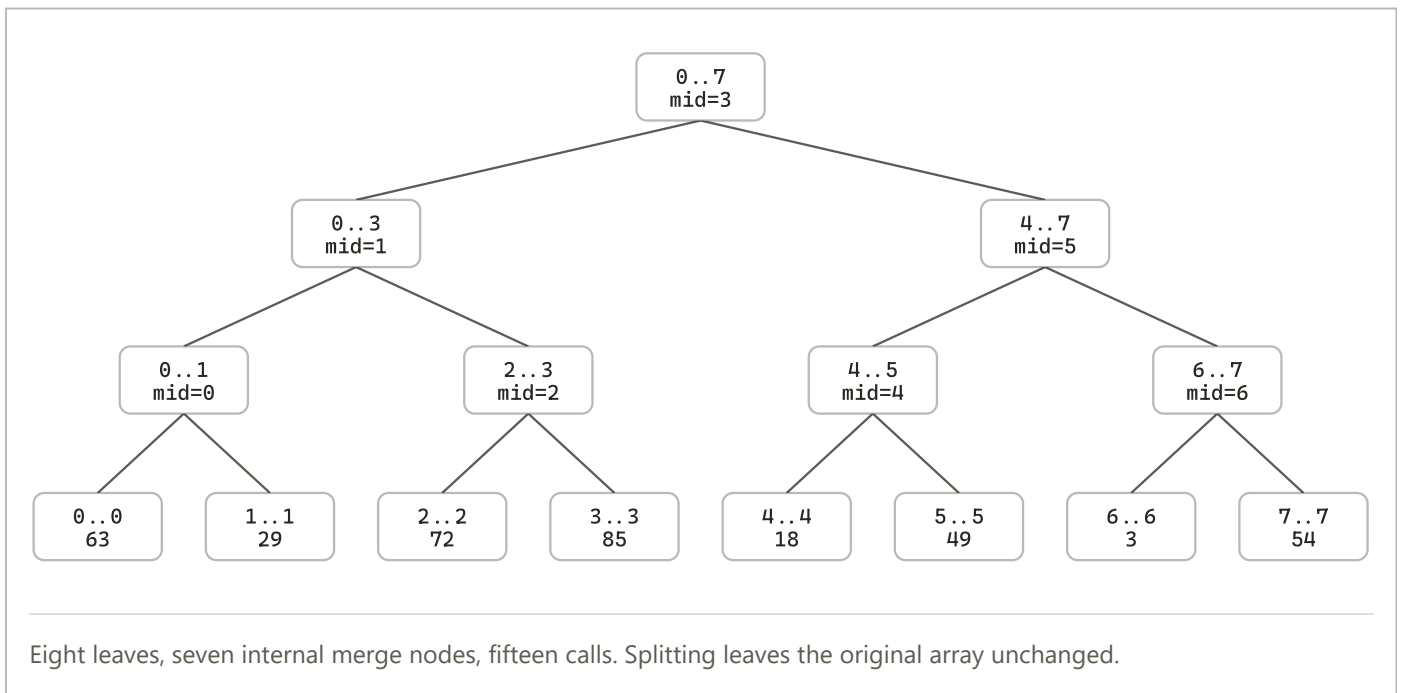
Write it from scratch

- State the inclusive base case.
- Calculate a midpoint that shrinks both ranges.
- Sort left, sort right, then merge.

- Track both input pointers and temp output.
- Choose one front value; advance only its source.
- Copy both possible leftover suffixes.
- Copy temp back with start + k.

Recursion, calls and returns

The primary implementation is already recursive. Its 15 calls form a tree, but Java executes them depth-first and retains only a path on the stack.



The live stack at the first base case

CALL STACK - NEWEST FRAME AT THE TOP

mergeSort(0, 0)

Base case: return

mergeSort(0, 1)

Waiting: Call left 0..0, then right 1..1, then merge

mergeSort(0, 3)

Waiting: Call left 0..1, then right 2..3, then merge

mergeSort(0, 7)

Waiting: Call left 0..3, then right 4..7, then merge

Only four frames are live at this moment: root, left half, left pair and first leaf. The other eleven tree nodes are not simultaneous stack frames.

The base-case frame returns and disappears. Its parent then calls the right leaf, merges the pair, and returns to its own parent. A parent resumes exactly where its child call suspended it.

Every call, with its midpoint

CALL #	DEPTH	RANGE	MID	NEXT WORK
1	0	0..7	3	left 0..3, right 4..7, then merge
2	1	0..3	1	left 0..1, right 2..3, then merge
3	2	0..1	0	left 0..0, right 1..1, then merge
4	3	0..0	-	Base case: return
5	3	1..1	-	Base case: return
6	2	2..3	2	left 2..2, right 3..3, then merge
7	3	2..2	-	Base case: return
8	3	3..3	-	Base case: return
9	1	4..7	5	left 4..5, right 6..7, then merge
10	2	4..5	4	left 4..4, right 5..5, then merge
11	3	4..4	-	Base case: return

CALL #	DEPTH	RANGE	MID	NEXT WORK
12	3	5..5	-	Base case: return
13	2	6..7	6	left 6..6, right 7..7, then merge
14	3	6..6	-	Base case: return
15	3	7..7	-	Base case: return

For 0..3, calls visit 0..1 and its leaves, then merge 0..1, then visit 2..3 and its leaves, then merge 2..3, then merge 0..3. Merges are NOT executed level by level.

Inside a merge: pointers, temporary output and copy-back

The completed-return view answers *which merge returns next?* This view answers *how does a merge construct its result?*

The final merge: every selection, leftover and copy-back

Start with two sorted runs

Left run 0..3; left = 0

29

63

72

85

0

1

2

3

left

Right run 4..7; right = 4

3

18

49

54

4

5

6

7

right

Temporary output; out = 0

.

.

.

.

.

.

.

.

0

1

2

3

4

5

6

7

Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced. Temporary output indexes begin at 0.

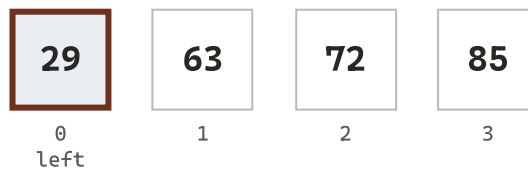
CSI247 · Merge Sort

38 / 87

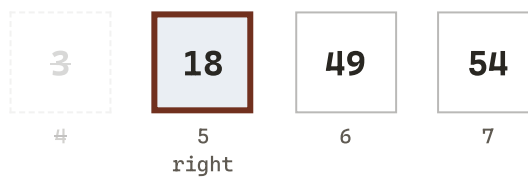
left = 0, right = 4, out = 0. Both input ranges are sorted.

Write temp[out] = 3

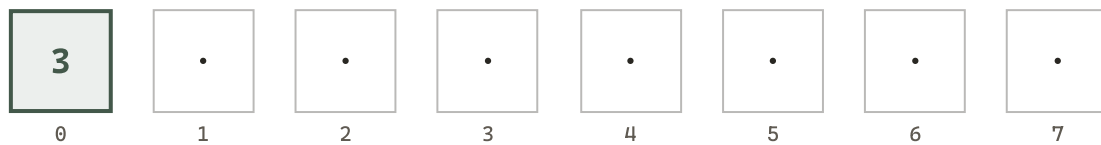
Left run 0..3; left = 0



Right run 4..7; right = 5



Temporary output; out = 1

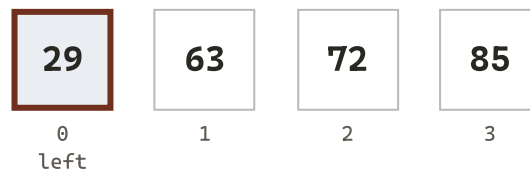


Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced.
Temporary output indexes begin at 0.

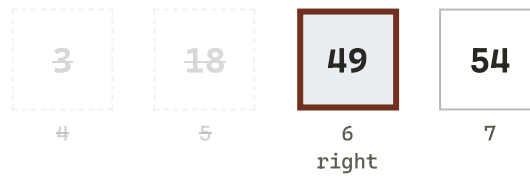
$29 \leq 3 \rightarrow \text{false}$; take right and advance only its pointer.

Write temp[1] = 18

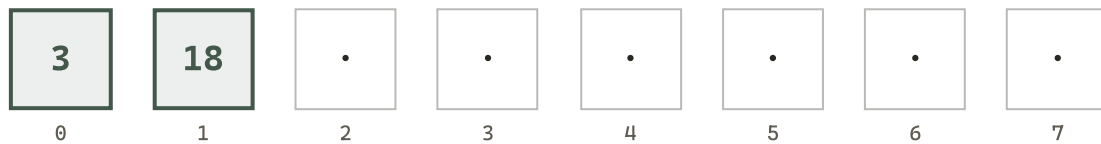
Left run 0..3; left = 0



Right run 4..7; right = 6



Temporary output; out = 2

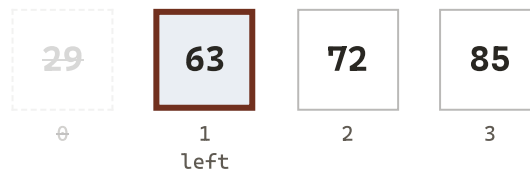


Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced.
Temporary output indexes begin at 0.

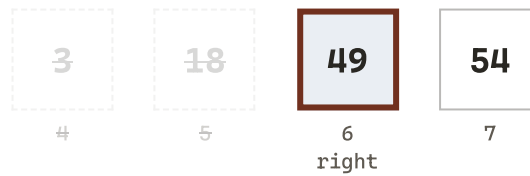
29 <= 18 → false; take right and advance only its pointer.

Write temp[2] = 29

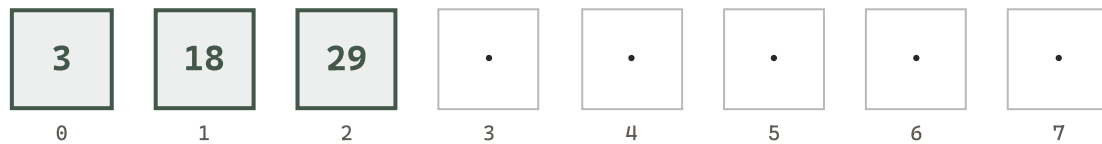
Left run 0..3; left = 1



Right run 4..7; right = 6



Temporary output; out = 3

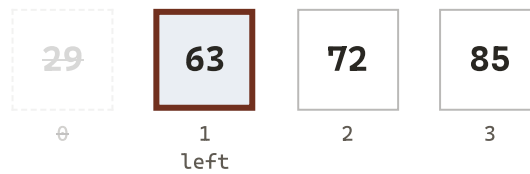


Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced.
Temporary output indexes begin at 0.

29 <= 49 → true; take left and advance only its pointer.

Write temp[3] = 49

Left run 0..3; left = 1



Right run 4..7; right = 7



Temporary output; out = 4

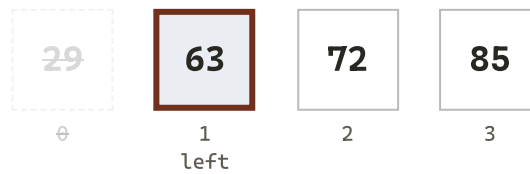


Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced.
Temporary output indexes begin at 0.

63 <= 49 → false; take right and advance only its pointer.

Write temp[4] = 54

Left run 0..3; left = 1



Right run 4..7; right = 8



Temporary output; out = 5



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced.
Temporary output indexes begin at 0.

63 <= 54 → false; take right and advance only its pointer.

Copy left leftover 63

Left run 0..3; left = 2

29	63	72	85
0	1	2	3
		left	

Right run 4..7; right = 8

3	18	49	54				
4	5	6	7				

Temporary output; out = 6

3	18	29	49	54	63	.	.
0	1	2	3	4	5	6	7

Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced.
Temporary output indexes begin at 0.

Right run is exhausted. Copy a remaining left value without a key comparison.

Copy left leftover 72

Left run 0..3; left = 3

29	63	72	85
0	1	2	3

left

Right run 4..7; right = 8

3	18	49	54
4	5	6	7

Temporary output; out = 7

3	18	29	49	54	63	72	.
0	1	2	3	4	5	6	7

Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced.
Temporary output indexes begin at 0.

Right run is exhausted. Copy a remaining left value without a key comparison.

Copy left leftover 85

Left run 0..3; left = 4



Right run 4..7; right = 8



Temporary output; out = 8



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced.
Temporary output indexes begin at 0.

Right run is exhausted. Copy a remaining left value without a key comparison.

Copy back arr[0] = 3

Left run 0..3; left = 4



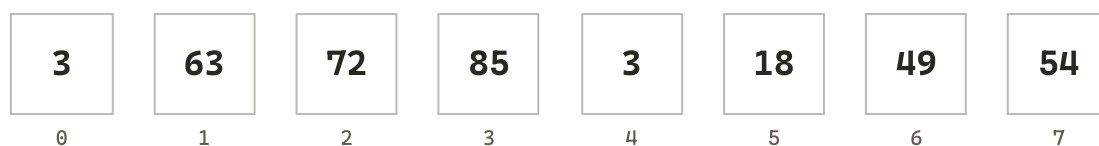
Right run 4..7; right = 8



Temporary output; out = 8



Shared array during copy-back



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced. Temporary output indexes begin at 0.

temp[0] is copied to arr[start + 0]. Unread source values were protected in temp.

Copy back arr[1] = 18

Left run 0..3; left = 4



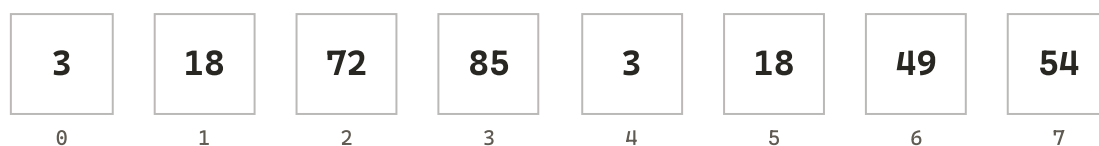
Right run 4..7; right = 8



Temporary output; out = 8



Shared array during copy-back



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced. Temporary output indexes begin at 0.

temp[1] is copied to arr[start + 1]. Unread source values were protected in temp.

Copy back $\text{arr}[2] = 29$

Left run 0..3; left = 4



Right run 4..7; right = 8



Temporary output; out = 8



Shared array during copy-back



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced.
Temporary output indexes begin at 0.

$\text{temp}[2]$ is copied to $\text{arr}[\text{start} + 2]$. Unread source values were protected in temp.

Copy back `arr[3] = 49`

Left run 0..3; left = 4



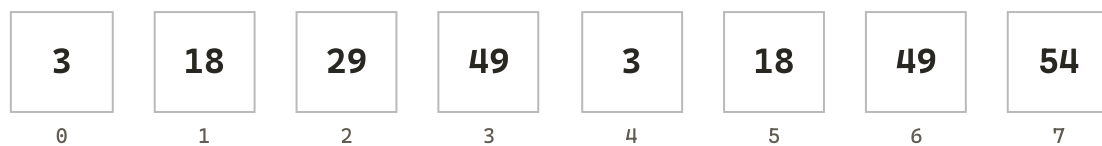
Right run 4..7; right = 8



Temporary output; out = 8



Shared array during copy-back



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced. Temporary output indexes begin at 0.

`temp[3]` is copied to `arr[start + 3]`. Unread source values were protected in `temp`.

Copy back arr[4] = 54

Left run 0..3; left = 4



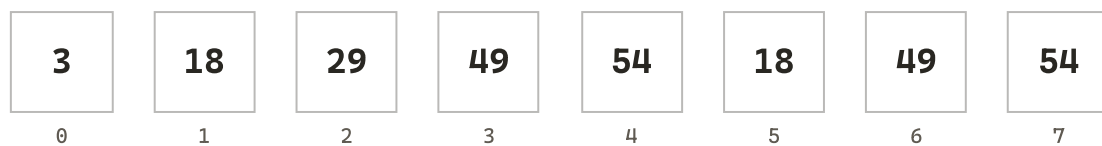
Right run 4..7; right = 8



Temporary output; out = 8



Shared array during copy-back



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced.
Temporary output indexes begin at 0.

temp[4] is copied to arr[start + 4]. Unread source values were protected in temp.

Copy back $\text{arr}[5] = 63$

Left run 0..3; left = 4



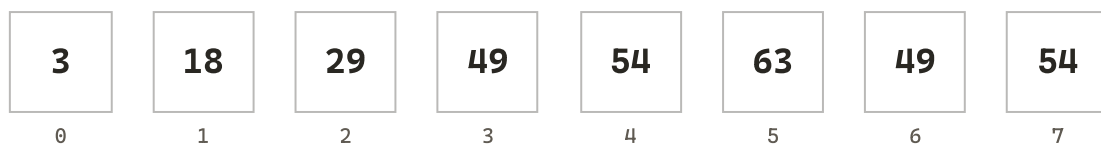
Right run 4..7; right = 8



Temporary output; out = 8



Shared array during copy-back



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced.
Temporary output indexes begin at 0.

$\text{temp}[5]$ is copied to $\text{arr}[\text{start} + 5]$. Unread source values were protected in temp.

Copy back $\text{arr}[6] = 72$

Left run 0..3; left = 4



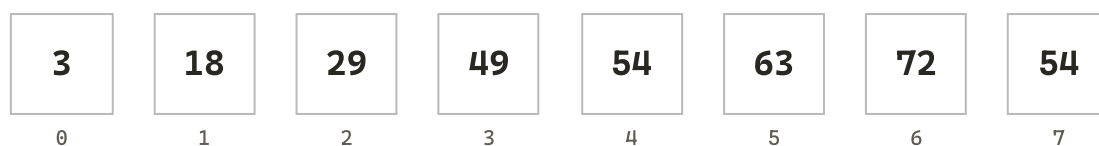
Right run 4..7; right = 8



Temporary output; out = 8



Shared array during copy-back



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced.
Temporary output indexes begin at 0.

$\text{temp}[6]$ is copied to $\text{arr}[\text{start} + 6]$. Unread source values were protected in temp.

Copy back arr[7] = 85

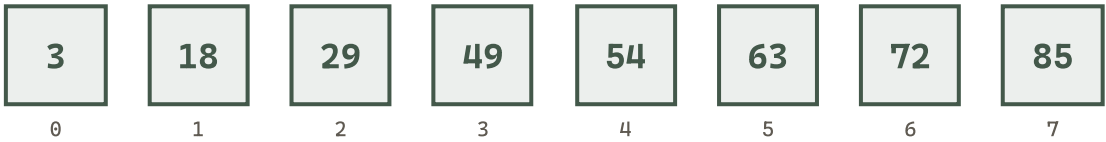
Left run 0..3; left = 4



Right run 4..7; right = 8



Temporary output; out = 8



Shared array during copy-back



Crossed values have been consumed; pointers may reach one-past-the-run-end without being dereferenced. Temporary output indexes begin at 0.

temp[7] is copied to arr[start + 7]. Unread source values were protected in temp.

▼ Full trace table - select a row to jump to its scene

STEP	LEFT (ABS OLUT E)	RIGH T (ABS OLUT E)	OUT	OPERATION	TEMP
1	0	4	0	left = 0, right = 4, out = 0. Both input ranges are sorted.	[]

STEP	LEFT (ABS OLUT E)	RIGH T (ABS OLUT E)	OUT	OPERATION	TEMP
2	0	5	1	29 <= 3 → false; take right and advance only its pointer.	[3]
3	0	6	2	29 <= 18 → false; take right and advance only its pointer.	[3,18]
4	1	6	3	29 <= 49 → true; take left and advance only its pointer.	[3,18,29]
5	1	7	4	63 <= 49 → false; take right and advance only its pointer.	[3,18,29,49]
6	1	8	5	63 <= 54 → false; take right and advance only its pointer.	[3,18,29,49,54]
7	2	8	6	Right run is exhausted. Copy a remaining left value without a key comparison.	[3,18,29,49,54,63]
8	3	8	7	Right run is exhausted. Copy a remaining left value without a key comparison.	[3,18,29,49,54,63,72]
9	4	8	8	Right run is exhausted. Copy a remaining left value without a key comparison.	[3,18,29,49,54,63,72,85]
10	4	8	8	temp[0] is copied to arr[start + 0]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]
11	4	8	8	temp[1] is copied to arr[start + 1]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]
12	4	8	8	temp[2] is copied to arr[start + 2]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]
13	4	8	8	temp[3] is copied to arr[start + 3]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]
14	4	8	8	temp[4] is copied to arr[start + 4]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]

STEP	LEFT (ABS OLUT E)	RIGH T (ABS OLUT E)	OUT	OPERATION	TEMP
1 5	4	8	8	temp[5] is copied to arr[start + 5]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]
1 6	4	8	8	temp[6] is copied to arr[start + 6]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]
1 7	4	8	8	temp[7] is copied to arr[start + 7]. Unread source values were protected in temp.	[3,18,29,49,54,63,72,85]

Final merge: 5 key comparisons, 3 leftover copies, 8 temp writes, then 8 copy-back writes. Loop-bound checks are not counted as key comparisons.

The temporary cells protect every unread value. The output offset start + k is essential when merging a subrange that does not begin at 0.

A complete runnable Java example

Save as MergeDemo.java. The code includes the method(s) and a main entry point.

```
import java.util.Arrays;

public class MergeDemo {
    public static void mergeSort(int[] arr, int start, int end) {
        // Inclusive range of zero or one value: stop before splitting.
        if (start ≥ end) return;
        // Split INDEX RANGES. This statement does not move array values.
        int mid = start + (end - start) / 2;
        mergeSort(arr, start, mid);    // Finish the entire left subtree first.
        mergeSort(arr, mid + 1, end);  // Then finish the entire right subtree.
        // Both halves are now sorted: the merge precondition is satisfied.
        merge(arr, start, mid, end);
    }

    private static void merge(int[] arr, int start, int mid, int end) {
        // Protect unread input: never merge directly over unprotected values.
        int[] temp = new int[end - start + 1];
        int left = start;    // First unread value of left run start..mid.
        int right = mid + 1; // First unread value of right run mid+1..end.
        int out = 0;         // Next free position in temp (starts at zero).
        while (left ≤ mid && right ≤ end) {
            // Left-first ties preserve the order of equal records.
            if (arr[left] ≤ arr[right]) {
                temp[out] = arr[left]; left++;
            } else {
                temp[out] = arr[right]; right++;
            }
            out++; // Exactly one output value was written.
        }
        // One run is exhausted. Its partner's suffix is already sorted.
        while (left ≤ mid) { temp[out++] = arr[left++]; }
        while (right ≤ end) { temp[out++] = arr[right++]; }
        // Offset matters: temp[0] belongs at arr[start], not always arr[0].
        for (int k = 0; k < temp.length; k++) arr[start + k] = temp[k];
        // First call: mergeSort(arr, 0, arr.length - 1).
    }

    public static void main(String[] args) {
        int[] values = {63, 29, 72, 85, 18, 49, 3, 54};
    }
}
```

```
mergeSort(values, 0, values.length - 1);
System.out.println(Arrays.toString(values));
}
}
```

Expected console output

[3, 18, 29, 49, 54, 63, 72, 85]

Sorting / Merge sort · Properties

Properties

BEST TIME

$O(n \log n)$

AVERAGE TIME

$O(n \log n)$

WORST TIME

$O(n \log n)$

EXTRA SPACE

$O(n)$

STABLE

Yes*

IN-PLACE

No

ADAPTIVE

No

TYPE

Divide and conquer

* stable because the merge takes the *left* value when the two fronts are equal ($\leq$).

Same time for every input

The array is cut in the same places whatever the values are, and every level merges every value. A sorted, reversed or random array all cost $O(n \log n)$. That predictability is merge sort's main selling point.

Stable

Equal values keep their original order, *provided* the merge uses $\leq$ so ties are taken from the left run (which contains the earlier values). Changing it to $<$ silently loses stability.

Not in-place

The merge copies both halves into temporary arrays before writing back, so extra memory proportional to n is needed: $O(n)$. This is the price paid for the guaranteed speed.

Not adaptive

The standard version cannot take advantage of existing order, as the sorted example showed.

Comparison-based

For comparison-based sorting, $O(n \log n)$ is the best possible worst case, and merge sort reaches it.

Sorting / Merge sort · Why those complexities occur

Why those complexities occur

Time · $O(n \log n)$, always $\log_2 n$ levels, and each level merges all n values once.	Space · $O(n)$ The merge needs temporary copies of the halves, up to n values at a time.
---	---

Count the levels, then count the work per level

How many times can you halve 8 before you reach 1? Three: $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$. In general $\log_2 n$ times. That is the number of rows of merging.

0 cuts		8 per piece
1 cut		4 per piece
2 cuts		2 per piece
3 cuts		1 per piece

Each row merges every value exactly once (the boxes in a row together hold all n values), so one row costs about n steps. Rows $\times$ work per row:

$$n \text{ values} \times \log_2 n \text{ levels} = n \log_2 n \text{ steps}$$

You can also write this as a recurrence: sorting n values costs two sorts of $n / 2$ plus one merge of n , that is $T(n) = 2 \cdot T(n / 2) + n$, and unrolling it gives the same $n \log_2 n$.

N (VALUES)	LEVELS [$\log_2 N$]	MERGE SORT $\approx$ N·LEVELS	A QUADRATIC SORT $\approx N^2$	QUADRATIC IS HOW MANY TIMES MORE
8	3	24	64	$\approx 2\times$
1,000	10	10,000	1,000,000	$\approx 100\times$

N (VALUES)	LEVELS [$\log_2 N$]	MERGE SORT $\approx$ N·LEVELS	A QUADRATIC SORT $\approx N^2$	QUADRATIC IS HOW MANY TIMES MORE
1,000,000	20	20,000,000	1,000,000,000,000	$\approx 50,000\times$

For a million values, merge sort takes about **20 million** steps and a quadratic sort about **a trillion**. Even if each step of merge sort cost ten times as much (because of copying), merge sort would still be thousands of times faster. This is why the sorting library of nearly every language uses a merge-sort relative.

Space

During one merge, two temporary arrays hold copies of the two halves: together at most n values. Merges happen one at a time, so the copies are discarded between them and the peak extra memory is $O(n)$, plus $O(\log n)$ for the call stack.

Sorting / Merge sort · When to use it

When to use it

A good choice Large arrays; when you need guaranteed $O(n \log n)$ whatever the data; when you need a stable sort (sorting records by one field while keeping the order of ties); when data is too big for memory and is sorted in chunks that are merged (external sorting); sorting linked lists.	A poor choice Very small arrays, where the overhead of recursion and copying costs more than a simple insertion sort; situations with very little spare memory ($O(n)$ extra space); data that is already nearly sorted, where insertion sort is almost free.
--	---

Real libraries combine the ideas: they use merge sort (or a close relative) for the big picture, and switch to **insertion sort** for pieces smaller than about 10 to 20 values, because on tiny arrays the simple sort wins. Knowing both lets you see why.

Sorting · Why sort data at all?

Why sort data at all?

Before the algorithms: what does sorted data give you that unsorted data does not, and what does it cost?

Everything in this chapter is about putting values in order. Before learning *how*, it is worth being clear about **why**, because it is a favourite exam question and because it explains why searching and sorting are taught together.

Short answer for a test: sorting data makes it easier and faster to find information in it. A sorted array can be searched with binary search, in $O(\log n)$ instead of $O(n)$.

What becomes easy once data is sorted

Take the nine values from the past paper. Unsorted they tell you nothing at a glance; sorted, a lot of questions answer themselves.



QUESTION	UNSORTED ARRAY	SORTED ARRAY
Smallest and largest value?	Look at all 9 cells	The first and last cell: 2 and 90

QUESTION	UNSORTED ARRAY	SORTED ARRAY
Median (middle) value?	Needs a lot of work	The middle cell: 50
Is a value present?	Linear search: up to 9 comparisons	Binary search: at most 4 probes
Where are the duplicates?	Anywhere	Next to each other (see below)
Merge two lists?	No shortcut	Merge them by zipping (that is merge sort's merge step)
Show a list to a person?	Hard to read	Alphabetical or numerical order, easy to scan

Duplicates end up side by side

Finding repeated values in [3, 1, 3, 2, 3] means comparing every pair. After sorting, [1, 2, 3, 3, 3], equal values are neighbours, so one pass comparing each cell with the next finds them all.

Sorting is not free: is it worth it?

Sorting costs about $n \log_2 n$ steps, and it only pays off if you search more than a few times. Suppose you have **1,000,000** values and will search for **1,000** different keys:

STRATEGY	WORK	TOTAL
Never sort: linear search every time	1,000 searches × up to 1,000,000 comparisons	1,000,000,000
Sort once, then binary search every time	sort ≈ 20,000,000 steps, then 1,000 searches × 20 probes	20,020,000

That is about **49× less work** after sorting. With a single search the answer flips: scanning once costs 1,000,000 and sorting first costs 20,000,000. The rule: *search once, scan; search often, sort first*.

▼ Quick check: Exam-style question 3(a), “What is the purpose of sorting data?”

To arrange data in a defined order (ascending or descending) so that information is easier and faster to find, for example by using binary search instead of scanning every element. Other valid points: duplicates become adjacent, minimum and maximum are at the ends, and sorted data is easier to read and merge.

Sorting

The sorting map

Every algorithm orders values, but each uses a different repeated action. The four are taught in this order: Bubble, Selection, Insertion, Merge.

ALGORITHM	REPEATED ACTION	TYPICAL TIME	BEST FIT
Bubble	Compare neighbours; swap wrong pairs	$O(n^2)$	Learning and tiny inputs
Selection	Find the smallest remaining value; place it next	$O(n^2)$	When few swaps matter
Insertion	Shift larger sorted values; insert a key	$O(n^2)$	Small or nearly sorted data
Merge	Divide into halves; merge sorted halves	$O(n \log n)$	Large data and predictable speed

Vocabulary behind every property tile

Comparison-based Order is decided only by asking questions such as $a < b$.	Best / average / worst The same algorithm on three arrangements of the same n values: already ordered, typical, and the most awkward.
In-place Values are rearranged inside the original array using only a few extra variables: $O(1)$ auxiliary space.	Auxiliary space Extra memory beyond the input array, such as the temporary copies merge sort makes.
Adaptive Does less work when the input is already or nearly sorted.	Stable Equal values keep their original relative order.

Why stability matters

Sort these students by mark. Ana and Ben both scored 70, and Ana was listed first.

Start

70	70	65
ANA	BEN	CY

Stable sort result

65	70	70
CY	ANA	BEN

An unstable sort may give

65	70	70
CY	BEN	ANA

Both outputs are sorted by mark. Only the stable one keeps Ana before Ben, which matters when you sort by one field and want an earlier ordering by another field preserved.

Count the disorder: inversions

An **inversion** is a pair of positions whose values are in the wrong order: a larger value sits to the left of a smaller one. [4, 2, 5, 1, 3] has six: (4,2), (4,1), (4,3), (2,1), (5,1), (5,3).

ALGORITHM	WHAT IS COUNTED	ON [4, 2, 5, 1, 3]	LINK TO INVERSIONS
Bubble	adjacent swaps	6	Each swap of neighbours removes exactly one inversion.
Insertion	shifts	6	Each shift removes exactly one inversion.
Selection	swaps	2	At most $n - 1$ swaps; one swap can remove several inversions at once.

A fully reversed array of n values has $n(n - 1) / 2$ inversions. For [5, 4, 3, 2, 1] that is 10, which equals the 10 shifts in the reverse-order insertion example. This is the reason bubble and insertion sort are $O(n^2)$ in the worst case.

Sorting / Merge sort · Descending order

Descending order

To sort largest to smallest, change the comparison that decides which front value is taken: take the **left** value when it is *greater than or equal to* the right one.

Ascending

```
arr[cur++] = left[i] ≤ right[j]
    ? left[i++]
    : right[j++];
```

Descending

```
arr[cur++] = left[i] ≥ right[j]
    ? left[i++]
    : right[j++];
```

The splitting, the recursion, the base case and the leftovers loops do not change. Only the merge direction does, and note that it is still = that makes ties take the left value, which keeps the sort stable in both directions.

The final merge, both ways

Ascending merges [29, 63, 72, 85] and [3, 18, 49, 54] into [3, 18, 29, 49, 54, 63, 72, 85]. Descending first sorts each half the other way, then merges [85, 72, 63, 29] and [54, 49, 18, 3] into [85, 72, 63, 54, 49, 29, 18, 3].

	ASCENDING	DESCENDING
Result	[3, 18, 29, 49, 54, 63, 72, 85]	[85, 72, 63, 54, 49, 29, 18, 3]
Calls / merges	15 / 7	15 / 7
Comparisons	14	15
Values written	24	24

The structure of the work is identical; the comparison count can differ slightly because it depends on how the runs interleave.

Sorting / Merge sort · Worked examples

Worked examples

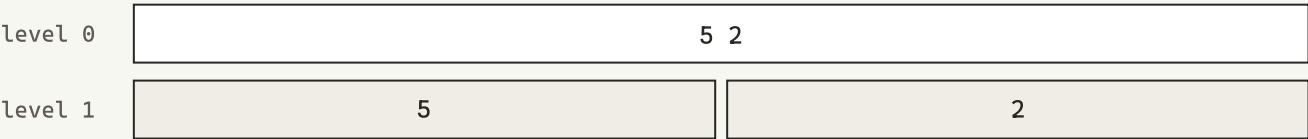
Six arrays: the smallest case, an odd length, sorted input, reversed input, duplicates and the nine values from the past paper.

Two values · [5, 2]

The smallest interesting case: one split, two base cases, one merge.



How the array is split (box width is proportional to size):



Merges, in the order they happen (3 calls, 1 merges):

#	MERGE CALL	LEFT RUN	RIGHT RUN	MERGED RESULT	COMPARISONS
1	merge(arr, 0, 0, 1)	[5]	[2]	[2, 5]	1
Total	1 merges			[2, 5]	1

3 calls and 1 merge. The two singles are compared once: $5 \leq 2$ is false, so the 2 goes first.

An odd number of values · [5, 2, 9, 1, 7]

Five values cannot be halved evenly: the left half gets three and the right half two, and the left three split again as 2 + 1.

5	2	9	1	7
0	1	2	3	4

How the array is split (box width is proportional to size):

level 0	5 2 9 1 7			
level 1	5 2 9		1 7	
level 2	5 2	9	1	7
level 3	5	2	9	1 7

Merges, in the order they happen (9 calls, 4 merges):

#	MERGE CALL	LEFT RUN	RIGHT RUN	MERGED RESULT	COMPARISONS
1	merge(arr, 0, 0, 1)	[5]	[2]	[2, 5]	1
2	merge(arr, 0, 1, 2)	[2, 5]	[9]	[2, 5, 9]	2
3	merge(arr, 3, 3, 4)	[1]	[7]	[1, 7]	1
4	merge(arr, 0, 2, 4)	[2, 5, 9]	[1, 7]	[1, 2, 5, 7, 9]	4
Total	4 merges			[1, 2, 5, 7, 9]	8

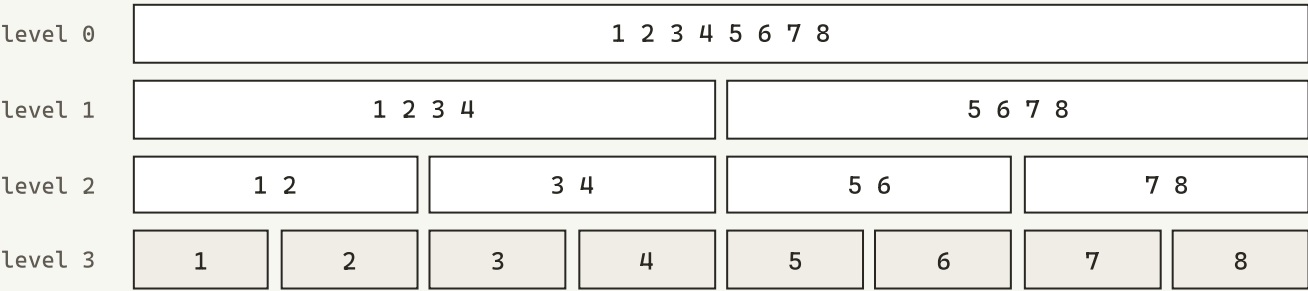
9 calls, 4 merges, 8 comparisons. Uneven halves are normal and the algorithm needs no special case for them.

Already sorted · [1, 2, 3, 4, 5, 6, 7, 8]

No disorder at all: does merge sort notice?



How the array is split (box width is proportional to size):



Merges, in the order they happen (15 calls, 7 merges):

#	MERGE CALL	LEFT RUN	RIGHT RUN	MERGED RESULT	COMPARISONS
1	merge(arr, 0, 0, 1)	[1]	[2]	[1, 2]	1
2	merge(arr, 2, 2, 3)	[3]	[4]	[3, 4]	1
3	merge(arr, 0, 1, 3)	[1, 2]	[3, 4]	[1, 2, 3, 4]	2
4	merge(arr, 4, 4, 5)	[5]	[6]	[5, 6]	1
5	merge(arr, 6, 6, 7)	[7]	[8]	[7, 8]	1
6	merge(arr, 4, 5, 7)	[5, 6]	[7, 8]	[5, 6, 7, 8]	2
7	merge(arr, 0, 3, 7)	[1, 2, 3, 4]	[5, 6, 7, 8]	[1, 2, 3, 4, 5, 6, 7, 8]	4
Total	7 merges			[1, 2, 3, 4, 5, 6, 7, 8]	12

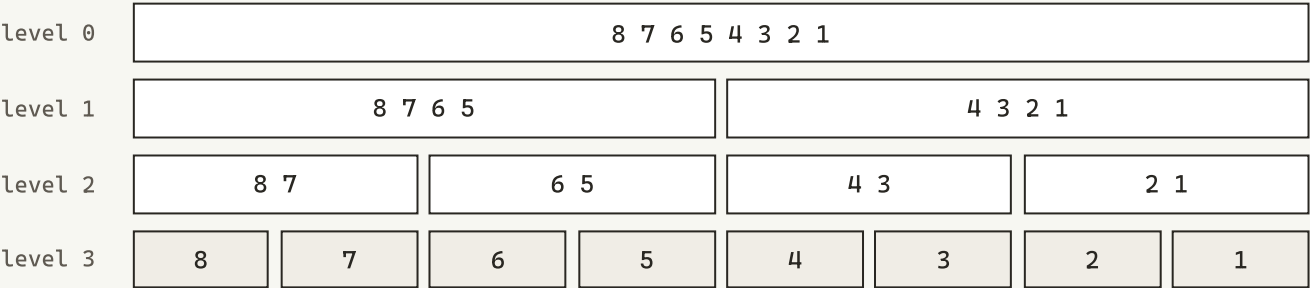
12 comparisons, compared with 14 for the unsorted example. A little cheaper (a run empties early), but the algorithm still makes every call and every merge. It is not adaptive.

Reverse order · [8, 7, 6, 5, 4, 3, 2, 1]

The input that ruins bubble and insertion sort.



How the array is split (box width is proportional to size):



Merges, in the order they happen (15 calls, 7 merges):

#	MERGE CALL	LEFT RUN	RIGHT RUN	MERGED RESULT	COMPARISONS
1	merge(arr, 0, 0, 1)	[8]	[7]	[7, 8]	1
2	merge(arr, 2, 2, 3)	[6]	[5]	[5, 6]	1
3	merge(arr, 0, 1, 3)	[7, 8]	[5, 6]	[5, 6, 7, 8]	2
4	merge(arr, 4, 4, 5)	[4]	[3]	[3, 4]	1
5	merge(arr, 6, 6, 7)	[2]	[1]	[1, 2]	1
6	merge(arr, 4, 5, 7)	[3, 4]	[1, 2]	[1, 2, 3, 4]	2
7	merge(arr, 0, 3, 7)	[5, 6, 7, 8]	[1, 2, 3, 4]	[1, 2, 3, 4, 5, 6, 7, 8]	4
Total	7 merges			[1, 2, 3, 4, 5, 6, 7, 8]	12

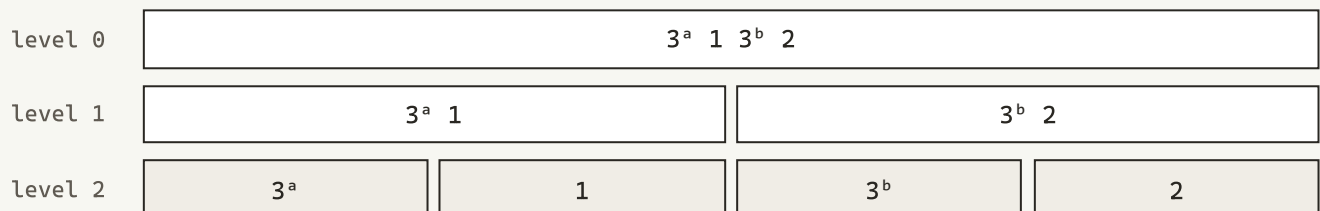
12 comparisons: the same order of magnitude as every other input. Merge sort has no bad input: $O(n \log n)$ in all cases.

Duplicates · $[3^a, 1, 3^b, 2]$

Equal values must keep their original order for the sort to be stable. Watch the merge of $[1, 3^a]$ with $[2, 3^b]$.



How the array is split (box width is proportional to size):



Merges, in the order they happen (7 calls, 3 merges):

#	MERGE CALL	LEFT RUN	RIGHT RUN	MERGED RESULT	COMPARISONS
1	<code>merge(arr, 0, 0, 1)</code>	$[3^a]$	$[1]$	$[1, 3^a]$	1
2	<code>merge(arr, 2, 2, 3)</code>	$[3^b]$	$[2]$	$[2, 3^b]$	1
3	<code>merge(arr, 0, 1, 3)</code>	$[1, 3^a]$	$[2, 3^b]$	$[1, 2, 3^a, 3^b]$	3
Total	3 merges			$[1, 2, 3^a, 3^b]$	5

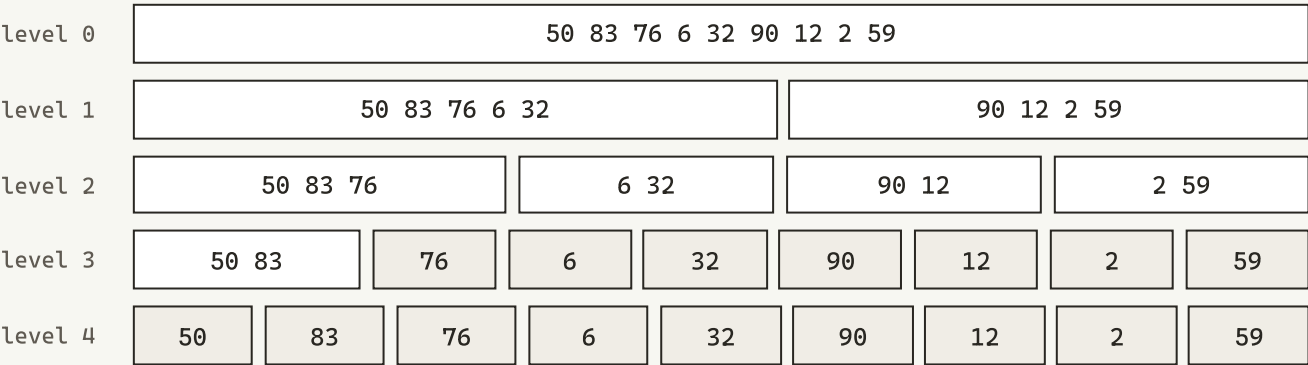
The result is $[1, 2, 3^a, 3^b]$: 3^a still comes before 3^b . When the two front values are equal (3^a vs 3^b) the $\leq$ takes the left one first. With $<$ it would take the right one and break stability.

The nine values from the past paper · $[50, 83, 76, 6, 32, 90, 12, 2, 59]$

Nine values: the first cut is $5 \mid 4$, the left five cut as $3 \mid 2$, and so on. Check each merge by merging by hand.

50	83	76	6	32	90	12	2	59
0	1	2	3	4	5	6	7	8

How the array is split (box width is proportional to size):



Merges, in the order they happen (17 calls, 8 merges):

#	MERGE CALL	LEFT RUN	RIGHT RUN	MERGED RESULT	COMPARISONS
1	merge(arr, 0, 0, 1)	[50]	[83]	[50, 83]	1
2	merge(arr, 0, 1, 2)	[50, 83]	[76]	[50, 76, 83]	2
3	merge(arr, 3, 3, 4)	[6]	[32]	[6, 32]	1
4	merge(arr, 0, 2, 4)	[50, 76, 83]	[6, 32]	[6, 32, 50, 76, 83]	2
5	merge(arr, 5, 5, 6)	[90]	[12]	[12, 90]	1
6	merge(arr, 7, 7, 8)	[2]	[59]	[2, 59]	1
7	merge(arr, 5, 6, 8)	[12, 90]	[2, 59]	[2, 12, 59, 90]	3
8	merge(arr, 0, 4, 8)	[6, 32, 50, 76, 83]	[2, 12, 59, 90]	[2, 6, 12, 32, 50, 59, 76, 83, 90]	8
Total	8 merges			[2, 6, 12, 32, 50, 59, 76, 83, 90]	19

17 calls, 8 merges, 19 comparisons. Always $n - 1 = 8$ merges for $n = 9$: each merge reduces the number of runs by one.

Sorting / Merge sort · Common mistakes

Common mistakes

- 1 Recursing on `mid` instead of `mid + 1` for the right half.**
The ranges must not overlap: left is `start .. mid`, right is `mid + 1 .. end`. Overlap causes endless recursion or wrong results.
- 2 Merging before the halves are sorted.**
`merge` must come *after* both recursive calls. Merge only works on two sorted runs.
- 3 Using `<` instead of `≤` in the merge.**
The sort still sorts but is no longer stable: equal values from the right run jump ahead of equal values from the left.
- 4 Forgetting the leftovers.**
When one run is exhausted, the remaining values of the other must still be copied. Without the two leftover loops the end of the array keeps stale values.
- 5 Copying the left half with `copyOfRange(arr, s, m)`.**
The end index is exclusive, so the left half needs `m + 1`: `copyOfRange(arr, s, m + 1)`.
- 6 Wrong base case.**
Stop when `start ≥ end` (zero or one value). Testing `start == end` only can loop forever on an empty range.
- 7 Writing the answer into the temporary arrays.**
The result must land back in `arr`, starting at index `start`, not index 0.
- 8 Expecting the merges to happen level by level.**
The recursive version merges depth-first: the whole left half is sorted before the right half is touched.

A test-ready summary

TEST ANSWER

Merge sort is a stable, comparison-based, divide-and-conquer sorting algorithm. It splits the array at the midpoint, recursively sorts the left and right halves, and then merges the two sorted halves into one sorted array by repeatedly taking the smaller front value (taking the left value on ties). A single value is the base case. Its time complexity is $O(n \log n)$ in the best, average and worst cases, because there are $\log_2 n$ levels and each level merges n values, but it needs $O(n)$ extra space and is not in-place. It is the right choice for large arrays and when a guaranteed running time or stability matters, and a poor choice when memory is very limited.

Practice

Guided question

Use merge sort on $[6, 3, 8, 4]$. List every call in order, then every merge in the order it happens.

▼ Reveal the guided answer

Calls (indentation = depth):

`mergeSort(arr, 0, 3) → mid = 1`

`mergeSort(arr, 0, 1) → mid = 0`

`mergeSort(arr, 0, 0) → one value: return`

`mergeSort(arr, 1, 1) → one value: return`

`mergeSort(arr, 2, 3) → mid = 2`

`mergeSort(arr, 2, 2) → one value: return`

`mergeSort(arr, 3, 3) → one value: return`

Merges:

#	MERGE CALL	LEFT RUN	RIGHT RUN	MERGED RESULT	COMPARISONS
1	merge(arr, 0, 0, 1)	[6]	[3]	[3, 6]	1
2	merge(arr, 2, 2, 3)	[8]	[4]	[4, 8]	1
3	merge(arr, 0, 1, 3)	[3, 6]	[4, 8]	[3, 4, 6, 8]	3
Total	3 merges			[3, 4, 6, 8]	5

Independent question

Show how [9, 4, 7, 3, 5] is split (remember the left half gets the extra value), then list the merges.

- ☐ Draw the split tree down to single values.
- ☐ Work out the merges in depth-first order: whole left side first.
- ☐ Write each merged result.
- ☐ It must take exactly $n - 1 = 4$ merges.

▼ Reveal the independent answer

level 0	9 4 7 3 5			
level 1	9 4 7		3 5	
level 2	9 4	7	3	5
level 3	9	4	7	3 5

#	MERGE CALL	LEFT RUN	RIGHT RUN	MERGED RESULT	COMPARISONS
1	merge(arr, 0, 0, 1)	[9]	[4]	[4, 9]	1
2	merge(arr, 0, 1, 2)	[4, 9]	[7]	[4, 7, 9]	2
3	merge(arr, 3, 3, 4)	[3]	[5]	[3, 5]	1
4	merge(arr, 0, 2, 4)	[4, 7, 9]	[3, 5]	[3, 4, 5, 7, 9]	3

#	MERGE CALL	LEFT RUN	RIGHT RUN	MERGED RESULT	COMPARISONS
Total	4 merges			[3, 4, 5, 7, 9]	7

Test-style question

List every merge made when merge sort sorts [50, 83, 76, 6, 32, 90, 12, 2, 59] ascending, and give the total number of comparisons.

▼ Reveal the test-style answer

#	MERGE CALL	LEFT RUN	RIGHT RUN	MERGED RESULT	COMPARISONS
1	merge(arr, 0, 0, 1)	[50]	[83]	[50, 83]	1
2	merge(arr, 0, 1, 2)	[50, 83]	[76]	[50, 76, 83]	2
3	merge(arr, 3, 3, 4)	[6]	[32]	[6, 32]	1
4	merge(arr, 0, 2, 4)	[50, 76, 83]	[6, 32]	[6, 32, 50, 76, 83]	2
5	merge(arr, 5, 5, 6)	[90]	[12]	[12, 90]	1
6	merge(arr, 7, 7, 8)	[2]	[59]	[2, 59]	1
7	merge(arr, 5, 6, 8)	[12, 90]	[2, 59]	[2, 12, 59, 90]	3
8	merge(arr, 0, 4, 8)	[6, 32, 50, 76, 83]	[2, 12, 59, 90]	[2, 6, 12, 32, 50, 59, 76, 83, 90]	8
Total	8 merges			[2, 6, 12, 32, 50, 59, 76, 83, 90]	19

The same array, four algorithms

Work done sorting [4, 2, 5, 1, 3] ascending. Only five values, but the pattern already shows each algorithm’s character.

ALGORITHM	COMPARISONS	DATA MOVEMENT	ARRAY WRITES (SAME UNIT)	WHAT THE NUMBERS SHOW
Bubble	10	6 swaps	12	One swap per inversion; each swap writes two cells.
Selection	10	2 swaps	4	Comparisons never shrink, but swaps are minimal.
Insertion	8	6 shifts	10	Comparisons stop early at the first failed test.
Merge	7	12 writes	12	Fewest comparisons on <i>this</i> input (on an already sorted array of 5, insertion needs 4 and merge 7); pays with temporary copies.

Counts come from the exact code in this guide: insertion stops at the left edge without an extra comparison, and merge counts only pairwise comparisons. “Data movement” mixes units on purpose, because that is how each algorithm is described; the last column converts them to one unit, writes into the array (a swap writes 2 cells, a shift 1, each insertion 1 more, and merge writes each value once per level it takes part in). Merge also copies halves into temporary arrays, which is not counted here.

Properties at a glance

ALGORITHM	BEST	AVERAGE	WORST	STABLE	IN-PLACE	EXTRA SPACE	ADAPTIVE
Bubble (with swapped flag)	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	Yes	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Yes	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	Yes	Yes	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	No	$O(n)$	No

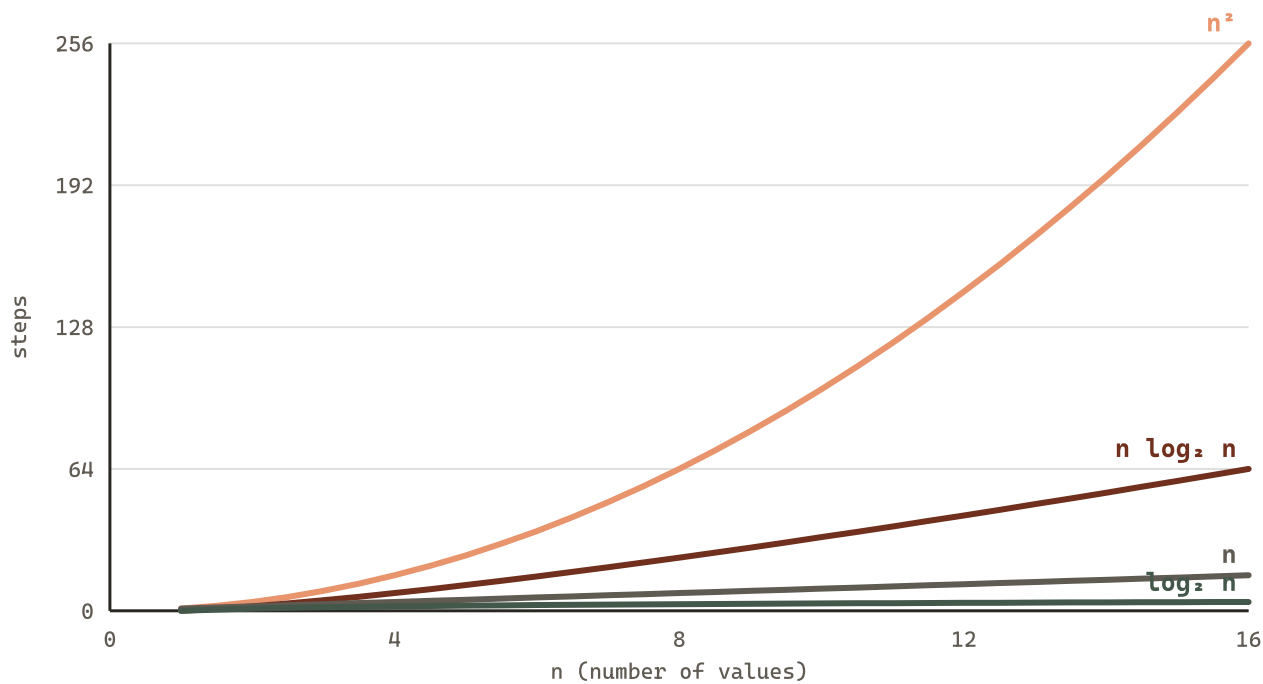
Stable: equal values keep their original order. Adaptive: faster when the input is already nearly sorted. Selection sort is stable only if implemented with shifting instead of a long-distance swap, which is not the version taught here.

Sorting & searching · Big-O growth side by side

Big-O growth side by side

The same four growth rates compared as a chart and as numbers, and the ranking behind Exam-style question 3(d)–(e).

Big-O answers one question: *when the input gets bigger, how fast does the number of steps grow?* It ignores small details (a multiplier of 2, a few extra steps) and keeps only the term that dominates when n is large. This chart plots the four growth rates that appear in this guide, for n from 0 to 16.



Read it from the bottom: $\log_2 n$ barely rises at all; n is a straight diagonal; $n \log_2 n$ bends upward a little; n^2 shoots away. At $n = 16$ the steps are about 4, 16, 64 and 256.

N (VALUES)	LOG ₂ N	N	N LOG ₂ N	N ²
10	3.3	10	33	100
100	6.6	100	664	10,000
1,000	10.0	1,000	9,965	1,000,000
1,000,000	19.9	1,000,000	19,931,568	1,000,000,000,000

A million values: $\log_2 n$ is 20; $n \log_2 n$ is 20 million; n^2 is a *trillion*. That last gap is why the choice of algorithm matters far more than how fast your computer is.

Ranking the algorithms in this guide (worst case)

ALGORITHM	JOB	WORST-CASE TIME	WHERE IT COMES FROM	EFFICIENCY RANK
Linear search	find a value	$O(n)$	may check every cell	3rd
Binary search	find a value (sorted data)	$O(\log n)$	halves the candidates each probe	1st: most efficient
Selection sort	sort	$O(n^2)$	$n(n-1)/2$ comparisons always	5th
Bubble sort	sort	$O(n^2)$	$n(n-1)/2$ comparisons in the worst case	5th
Insertion sort	sort	$O(n^2)$	$n(n-1)/2$ shifts in the worst case	5th
Merge sort	sort	$O(n \log n)$	$\log_2 n$ levels $\times$ n values per level	4th

Exam-style question 3(d)–(e) asks for the worst-case complexity of linear search, binary search, selection sort and bubble sort, and which of them is the most efficient. The answers: $O(n)$, $O(\log n)$, $O(n^2)$, $O(n^2)$, and **binary search** is the most efficient, because $\log n$ grows slowest.

Two traps on that question: (1) Binary search is $O(\log n)$, not $O(n^2)$: it does *not* have a loop inside a loop; each pass halves the work. (2) Comparing a searching algorithm with sorting algorithms is not quite fair, since binary search needs data that is already sorted. Say so if the question lets you; the expected answer is still binary search.

How to read a Big-O answer quickly

- **One loop over all the data** $\rightarrow O(n)$. Linear search.
- **A loop inside a loop over the data** $\rightarrow O(n^2)$. Bubble, selection and insertion sort.
- **Halve the problem each step** $\rightarrow O(\log n)$. Binary search.
- **Halve the problem *and* do n work at each level** $\rightarrow O(n \log n)$. Merge sort.

▼ **Quick check: put these in order from fastest to slowest growth: $O(n^2)$, $O(\log n)$, $O(n \log n)$, $O(n)$**

$O(\log n) < O(n) < O(n \log n) < O(n^2)$. Check with $n = 1,000$: about 10, 1,000, 10,000, 1,000,000.

More worked cases: edge cases, duplicates and descending order

Each case has input and result diagrams, evaluated decisions or merge inputs, exact counts, and a final state.

1. Nearly sorted

1

2

4

3

5

01234

↓ ascending merge sort

1

2

3

4

5

01234

The trace below connects the input diagram to the final result.

RETURN	RANGE	INPUTS	RESULT	KEY COMPARISONS
1	0..1	[1] + [2]	[1,2]	1
2	0..2	[1,2] + [4]	[1,2,4]	2
3	3..4	[3] + [5]	[3,5]	1
4	0..4	[1,2,4] + [3,5]	[1,2,3,4,5]	4

8 key comparisons; 12 copy-back writes.

2. Reverse order

5

4

3

2

1

01234

↓ ascending merge sort

1

2

3

4

5

01234

The trace below connects the input diagram to the final result.

RETURN	RANGE	INPUTS	RESULT	KEY COMPARISONS
1	0..1	[5] + [4]	[4,5]	1
2	0..2	[4,5] + [3]	[3,4,5]	1
3	3..4	[2] + [1]	[1,2]	1
4	0..4	[3,4,5] + [1,2]	[1,2,3,4,5]	2

5 key comparisons; 12 copy-back writes.

3. Duplicate values

2

2

1

3

0123

↓ ascending merge sort

1

2

2

3

0123

The trace below connects the input diagram to the final result.

RETURN	RANGE	INPUTS	RESULT	KEY COMPARISONS
1	0..1	[2] + [2]	[2,2]	1
2	2..3	[1] + [3]	[1,3]	1
3	0..3	[2,2] + [1,3]	[1,2,2,3]	3

5 key comparisons; 8 copy-back writes.

4. Uneven length

38

27

43

3

9

01234

↓ ascending merge sort

3

9

27

38

43

01234

The trace below connects the input diagram to the final result.

RETURN	RANGE	INPUTS	RESULT	KEY COMPARISONS
1	0..1	[38] + [27]	[27,38]	1
2	0..2	[27,38] + [43]	[27,38,43]	2
3	3..4	[3] + [9]	[3,9]	1
4	0..4	[27,38,43] + [3,9]	[3,9,27,38,43]	2

6 key comparisons; 12 copy-back writes.

5. Descending target

8

3

5

1

0

1

2

3

↓ descending merge sort

8

5

3

1

0

1

2

3

The trace below connects the input diagram to the final result.

RETURN	RANGE	INPUTS	RESULT	KEY COMPARISONS
1	0..1	[8] + [3]	[8,3]	1
2	2..3	[5] + [1]	[5,1]	1
3	0..3	[8,3] + [5,1]	[8,5,3,1]	3

5 key comparisons; 8 copy-back writes. Use `arr[left] >= arr[right]` to choose the left run. Both children must also be sorted descending; left-first equality still preserves stability.

Exam-style practice with worked answers

▼ A. Trace [8, 3, 5, 1] in descending order. Show every completed pass/merge. [8]

RETURN	LEFT	RIGHT	RESULT
1	[8]	[3]	[8,3]
2	[5]	[1]	[5,1]
3	[8,3]	[5,1]	[8,5,3,1]

Final result [8,5,3,1]. Use $arr[left] \geq arr[right]$ to choose the left run. Both children must also be sorted descending; left-first equality still preserves stability.

▼ B. Explain stability with tagged equal keys. [4]

On a tie between a left 2a and right 2b, $\leq$ selects 2a. Repeated left-first choices preserve original order.

▼ C. Give the time and space, then justify both. [5]

Best / average / worst: $O(n \log n)$. The standard implementation still splits and merges sorted input. Auxiliary arrays: $O(n)$ peak. The largest merge needs n temporary cells. Call stack: $O(\log n)$. Only one root-to-leaf path is active at once. Stable / in place / adaptive: Yes / no / no. Left-first ties preserve equals; the taught form allocates temp; no ordered-range shortcut.

▼ D. Debug the code before tracing it. [4]

MISTAKE	SYMPTOM	FIX
Right child starts at mid	Ranges overlap and may not shrink.	Use mid + 1.
Merge before both calls return	Input runs are not yet known to be sorted.	Call left, call right, then merge.

▼ E. What happens for [] and [7]? [2]

Both are already sorted. No key comparisons or value movements are required. Merge sort returns at its base case; the iterative methods never enter a pass.

▼ F. Merge [2, 9] and [4, 5, 8]. Show pointers and leftovers. [6]

Choose 2 (left), 4 (right), 5 (right), 8 (right), then copy leftover 9. Four key comparisons, one leftover, five output writes. Relative pointers end at $i=2, j=3$; result [2,4,5,8,9].

Topic quiz: test your understanding

Answer all 12 questions. Choose an option to see the reasoning; reset to retry. The printable answer key follows the questions.

1. What is the last valid index of an array of length 6? _____

☐ A. 5

☐ B. 6

☐ C. 7

2. What does $O(1)$ auxiliary space describe?

- ☐ A. Exactly one machine instruction
- ☐ B. The input contains one value
- ☐ C. Extra storage stays bounded as n grows

3. What is a loop invariant?

- ☐ A. A loop that never terminates
- ☐ B. A statement that remains true at a specified loop boundary
- ☐ C. A variable that never changes

4. What does stable sorting preserve?

- ☐ A. The original relative order of records with equal keys
- ☐ B. Every original array index
- ☐ C. The original order of unequal keys

5. What should a correct ascending sort do with `[]` and `[7]`?

- ☐ A. Throw an error for both
- ☐ B. Add a zero to each
- ☐ C. Leave both unchanged

6. What does splitting do to the array values?

- ☐ A. Swaps both halves
- ☐ B. Nothing; it divides index ranges
- ☐ C. Sorts each half immediately

7. When may a parent merge?

- ☐ A. After both child sorts return
- ☐ B. Before either child runs
- ☐ C. Immediately after computing mid

8. For inclusive start..end, where does the right child begin?

- ☐ A. mid
- ☐ B. end + 1
- ☐ C. mid + 1

9. What happens after one run is exhausted?

- ☐ A. Discard the remaining values
- ☐ B. Copy the other run's remaining values
- ☐ C. Compare against nonexistent values

10. Which side wins equality in a stable ascending merge?

- ☐ A. Left, using $\leq$
- ☐ B. Right, using $<$
- ☐ C. Either side, with no effect

11. Why write `arr[start + k] = temp[k]`?

- ☐ A. All merges start at index 0
- ☐ B. It avoids the need for temporary storage
- ☐ C. The destination subrange may begin after index 0

12. Time and peak auxiliary space of the taught merge sort?

- ☐ A. $O(\log n)$ time; $O(n^2)$ extra arrays
- ☐ B. $O(n \log n)$ time; $O(n)$ extra arrays
- ☐ C. $O(n^2)$ time; $O(1)$ extra arrays

▼ Answer key and explanations

1. **5.** Indexes start at 0, so the last index is $\text{length} - 1$.
2. **Extra storage stays bounded as n grows.** Auxiliary space counts extra working storage, excluding the input array.
3. **A statement that remains true at a specified loop boundary.** The invariant explains why each iteration preserves progress toward a correct result.
4. **The original relative order of records with equal keys.** Tags such as 2a and 2b let you check whether equal-key records crossed.
5. **Leave both unchanged.** Both inputs are already sorted; no key comparison is necessary.
6. **Nothing; it divides index ranges.** The recursion changes boundaries. Value copying happens during merging.
7. **After both child sorts return.** A merge relies on both input runs already being sorted.
8. **$\text{mid} + 1$.** The children must be disjoint and each must shrink toward the base case.
9. **Copy the other run's remaining values.** The leftovers are already ordered and require no further key comparisons.
10. **Left, using $\leq$.** Left-first ties preserve the original relative order of equal records.
11. **The destination subrange may begin after index 0.** Without start, a merge of a later subrange overwrites the beginning of the array.
12. **$O(n \log n)$ time; $O(n)$ extra arrays.** Each merge level handles n values; there are about $\log_2 n$ levels. Peak live storage is not the sum of all historical allocations.